

MEAN Full Stack

Amir Hussain
Cyber security trainer
&
Full Stack Developer





CHAPTER-1

Introduction



Definition

The word Web Development is made up of two words:

Web: It refers to websites, web pages or anything that works over the internet.

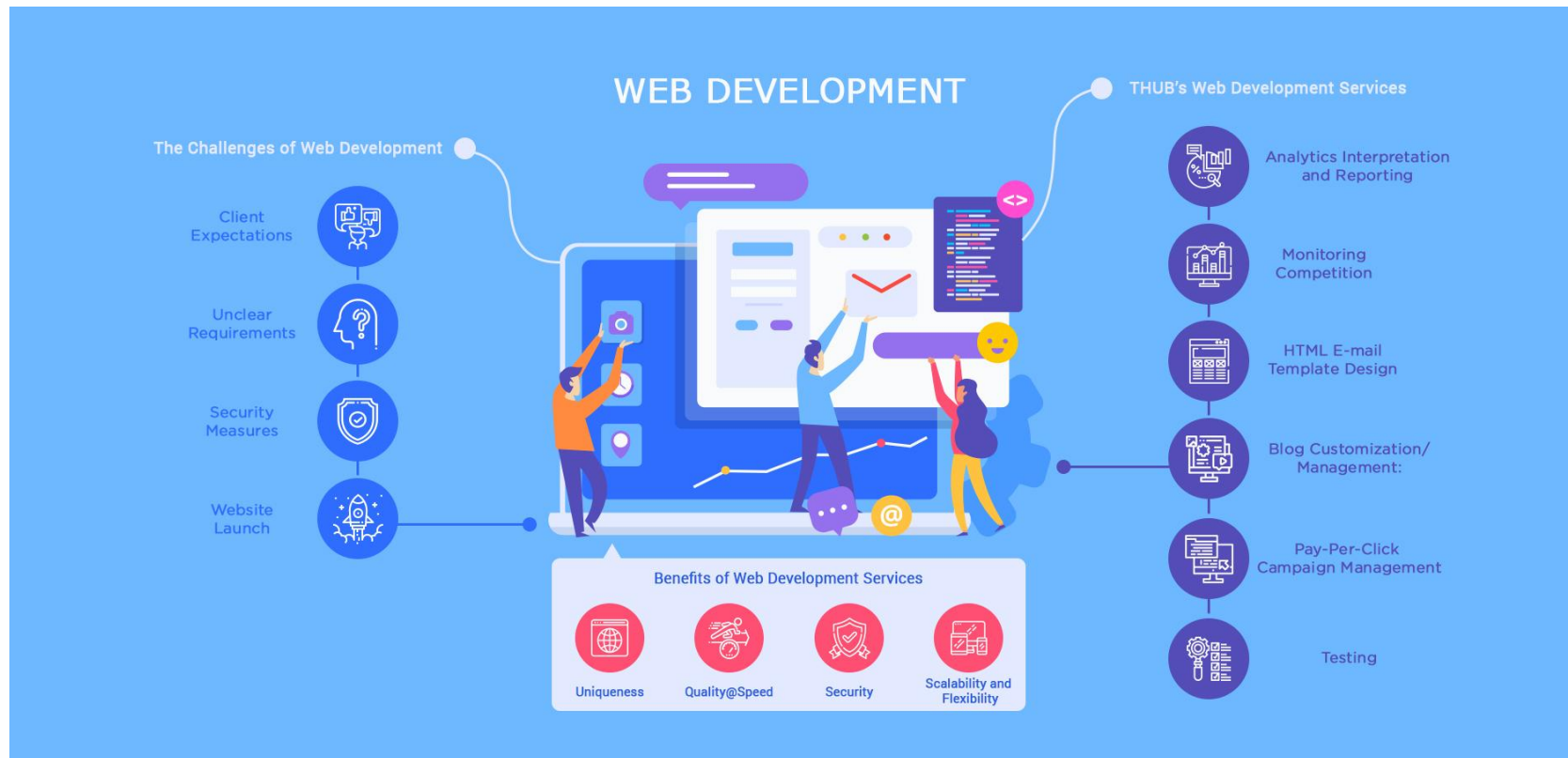
Development: Building the application from scratch.





What is Web Development?

- Web development refers to the building, creating, and maintaining of websites. It includes aspects such as web design, web publishing, web programming, and database management.
- It is the creation of an application that works over the internet i.e. websites.
- The term development is usually reserved for the actual construction of these things (that is to say, the programming of sites). The basic tools involved are programming languages called HTML (Hypertext Markup Language), CSS (Cascading Style Sheets), and JavaScript.





Classification

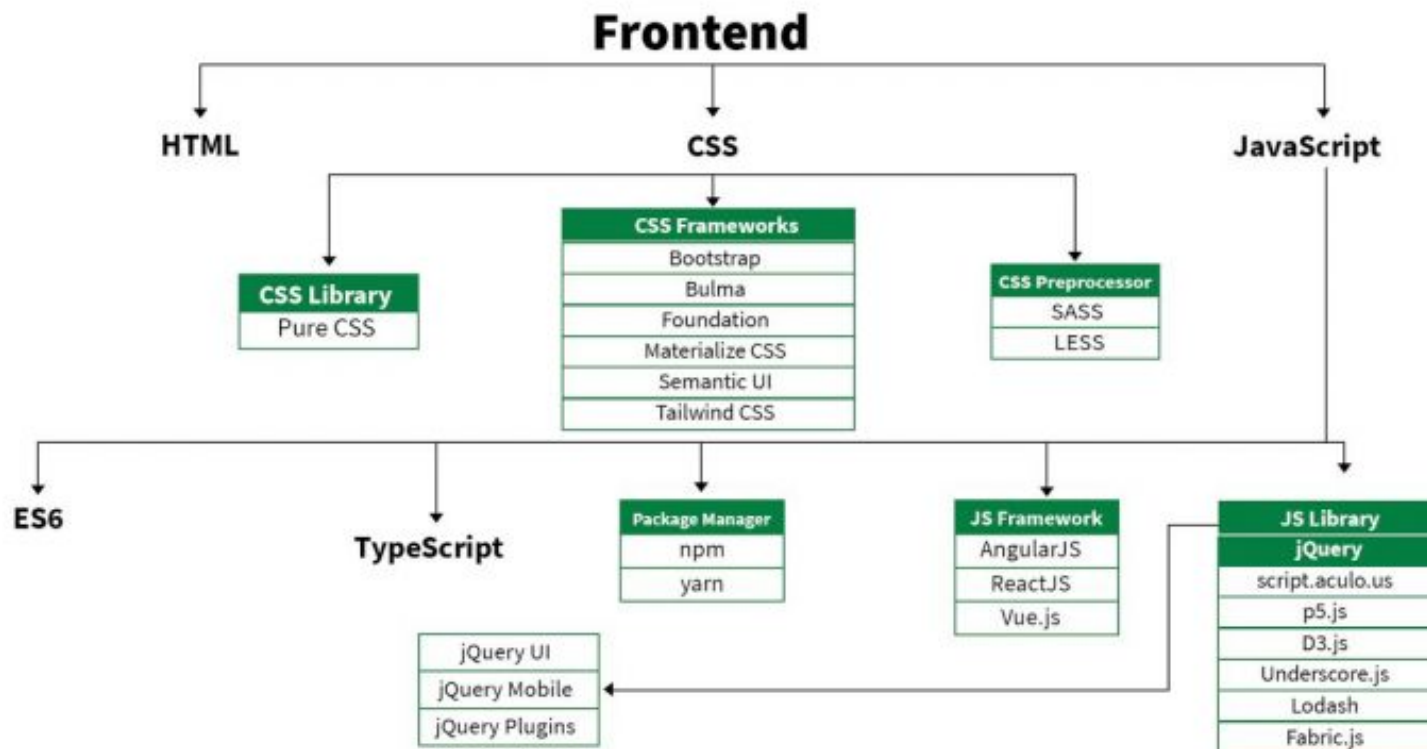
Web Development can be classified into two ways:

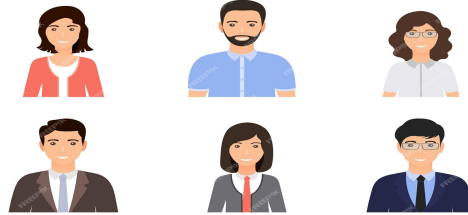
Frontend Development: The part of a website that the user interacts directly is termed as front end. It is also referred to as the 'client side' of the application.

Backend Development: Backend is the server side of a website. It is the part of the website that users cannot see and interact. It is the portion of software that does not come in direct contact with the users. It is used to store and arrange data.



Frontend Roadmap



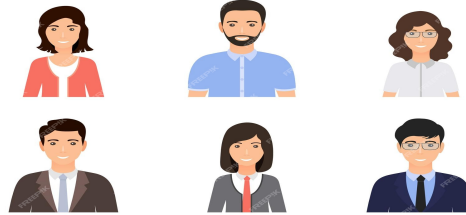


Frontend Roadmap

HTML: HTML stands for HyperText Markup Language. It is used to design the front end portion of web pages using markup language. It acts as a skeleton for a website since it is used to make the structure of a website.

CSS: Cascading Style Sheets fondly referred to as CSS is a simply designed language intended to simplify the process of making web pages presentable. It is used to style our website.

JavaScript: JavaScript is a scripting language used to provide a dynamic behavior to our website.



Frontend Roadmap

EXAMPLE OF A WEBSITE

Three layers of web design:

Structure

HTML markup

Style

CSS

Behavior

JavaScript



Frontend Roadmap

Bootstrap: Bootstrap is a free and open-source tool collection for creating responsive websites and web applications.

It is the most popular CSS framework for developing responsive, mobile-first websites. Nowadays, the websites are perfect for all the browsers (IE, Firefox, and Chrome) and for all sizes of screens (Desktop, Tablets, Phablets, and Phones).

Bootstrap 4

Bootstrap 5

Frontend Frameworks and Libraries:

AngularJS

React.js

VueJS

NextJS

Bootstrap

Material UI



Backend Development

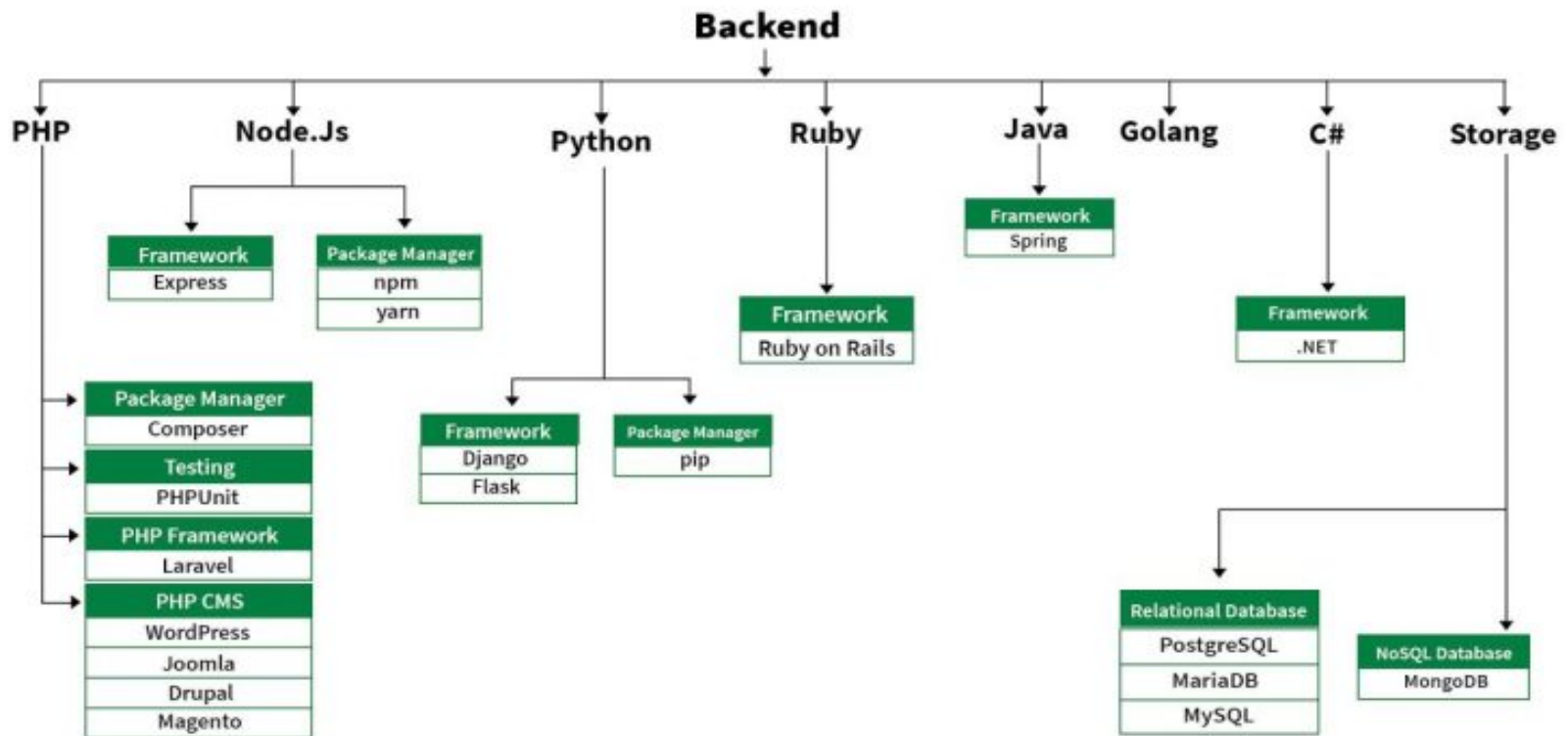
Backend Development controls what going on behind the scene of the web application.

Also Known as Server side development.

Backend usually consist three part.

1. Server
2. An Application
3. Database

Backend Roadmap





Backend Roadmap

PHP: PHP is a server-side scripting language designed specifically for web development.

Java: Java is one of the most popular and widely used programming language. It is highly scalable.

Python: Python is a programming language that lets you work quickly and integrate systems more efficiently.

Node.js: Node.js is an open source and cross-platform runtime environment for executing JavaScript code outside a browser.

Backend Roadmap

Back End Frameworks

The list of back end frameworks are:

Express,

Django,

NESTJs

Laravel,

Spring, etc.

Databases

No SQL:

- MongoDB
- BigTable
- Redis
- Cassandra
- HBase
- Neo4j
- CouchDB

SQL:

- MySQL
- Oracle
- PostgreSQL
- Microsoft SQL Server



What is MEAN stack?

The MEAN stack is a JavaScript-based framework for developing web applications. MEAN is named after MongoDB, Express, Angular, and Node, the four key technologies that make up the layers of the stack.

MongoDB — document database

Express(.js) — Node.js web framework

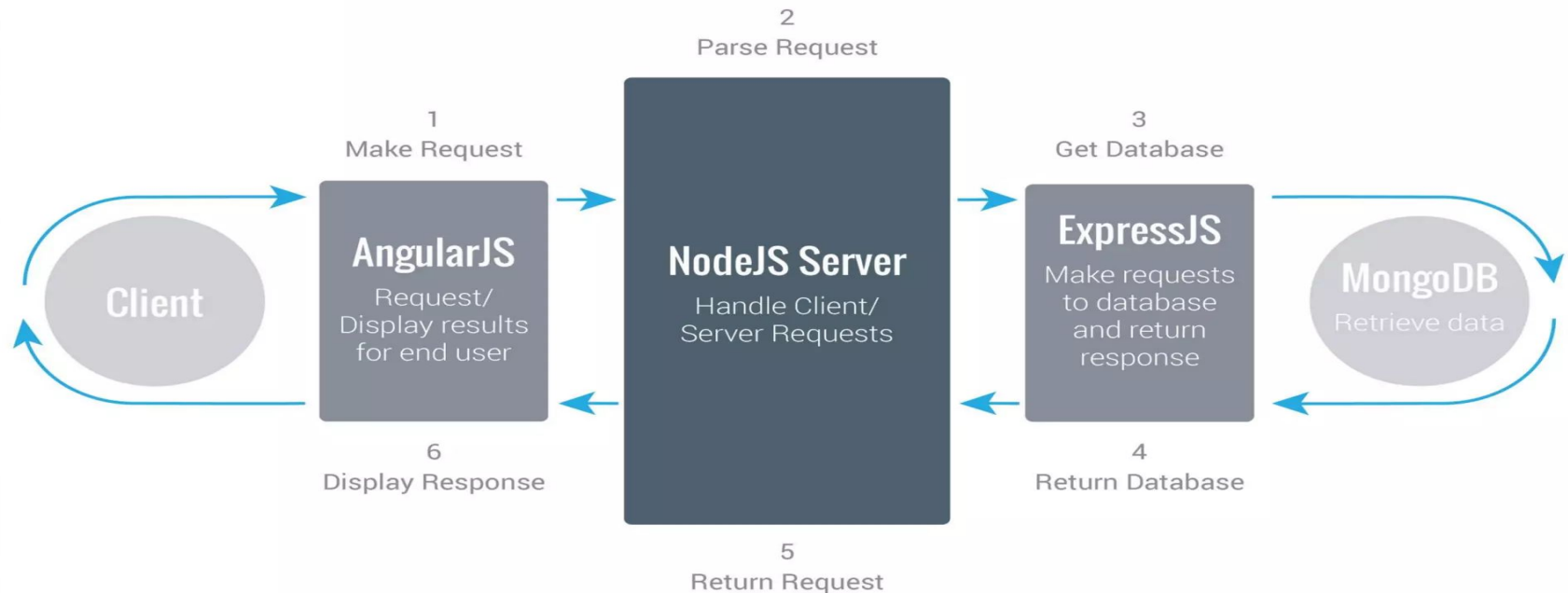
Angular(.js) — a client-side JavaScript framework

Node(.js) — the premier JavaScript web server

There are variations to the MEAN stack such as [MERN](#) (replacing Angular.js with React.js) and MEVN (using Vue.js). The MEAN stack is one of the most popular technology concepts for building web applications.

What is MEAN stack?

Processing model





Requirements?

1. Customer wants fast website and fast response.
2. No Page reloads
3. Mobile responsive
4. Etc.



Angular

Angular.js front end

At the very top of the MEAN stack is Angular.js, the self-styled “JavaScript MVW Framework” (MVW stands for “Model View and Whatever”).

Angular.js allows you to extend your HTML tags with metadata in order to create dynamic, interactive web experiences much more powerfully than, say, building them yourself with static HTML and JavaScript (or jQuery).

Angular has all of the bells and whistles you’d expect from a front-end JavaScript framework, including form validation, localization, and communication with your back-end service.



Express and NodeJs

The next level down is Express.js, running on a Node.js server.

Express.js calls itself a “fast, unopinionated, minimalist web framework for Node.js,” and that is indeed exactly what it is.

Express.js has powerful models for URL routing (matching an incoming URL with a server function), and handling HTTP requests and responses. By making XMLHttpRequests (XHRs), GETs, or POSTs from your Angular.js front end, you can connect to Express.js functions that power your application.

Those functions, in turn, use MongoDB’s Node.js drivers, either via callbacks or using promises, to access and update data in your MongoDB database.



MongoDB

If your application stores any data (user profiles, content, comments, uploads, events, etc.), then you're going to want a database that's just as easy to work with as Angular, Express, and Node.

That's where MongoDB comes in: JSON documents created in your Angular.js front end can be sent to the Express.js server, where they can be processed and (assuming they're valid) stored directly in MongoDB for later retrieval.



Setting up the environment

1. Install nodejs version(20.10.0) from <https://nodejs.org/en> and also visual code studio.
2. Open powershell (with internet connected)
3. Run: Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
4. Run: node --version
5. Run: npm -g install @angular/cli
6. Run: ng version
7. Open: visual code studio(create new folder) run following commands in visual code new>terminal
8. Run: ng new Yourfirstappname (it will take some time). then choose CSS(press enter)>SSR permission(y).
9. Run:cd ./Yourfirstappname
10. Run:ng serve
- 11.Open link created in your browser.



Conclusion

In today's Web development, a good page design is essential. A bad design will lead to the loss of visitors and that can lead to a loss of business. In general, a good page layout has to satisfy the basic elements of a good page design.

This includes color contrast, text organization, font selection, style of a page, page size, graphics used, and consistency. In order to create a well-designed page for a specific audience.

× DIGITAL LEARNING CONTENT

○



Parul[®] University



www.paruluniversity.ac.in

MEAN Full Stack

Amir Hussain
Cyber security trainer
&
Full Stack Developer





CHAPTER-2nd

Introduction



NoSQL database

NoSQL Database is used to refer a non-SQL or non-relational database.

It provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases.

NoSQL database doesn't use tables for storing data. It is generally used to store big data and real-time web applications.

NoSQL databases are a class of database management systems that differ from traditional relational databases in their data models, schemas, and scalability options



History behind the creation of NoSQL Databases

In the early 1970, Flat File Systems are used. Data were stored in flat files and the biggest problems with flat files are each company implement their own flat files and there are no standards. It is very difficult to store data in the files, retrieve data from files because there is no standard way to store data.

Then the relational database was created by E.F. Codd and these databases answered the question of having no standard way to store data. But later relational database also get a problem that it could not handle big data, due to this problem there was a need of database which can handle every types of problems then NoSQL database was developed.



Characteristics of NoSQL Databases

Flexibility: NoSQL databases are schema-less, allowing for flexible data models.

Scalability: Designed to scale horizontally to handle large amounts of data and traffic.

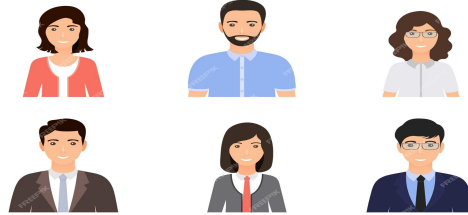
Variety of Data Models: Support for various data models like document-oriented, key-value, column-family, and graph databases.

High Performance: Optimized for read and write-intensive workloads.



Types of NoSQL Databases

- Document-Oriented: e.g. MongoDB
- Key-Value Stores: e.g. Redis
- Column-Family Stores: e.g. Apache Cassandra
- Graph Databases: e.g. Neo4j
- Object-Oriented Databases



Use Cases

Real-time Big Data Applications

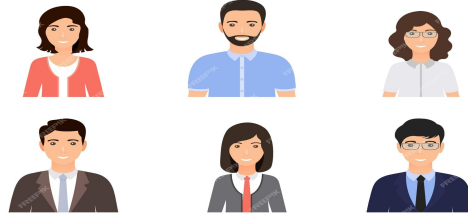
Content Management Systems (CMS)

Mobile App Development

Web App

IoT (Internet of Things) Applications

Social Media Analytics



MongoDB

- **Definition:** MongoDB is a popular open-source NoSQL document-oriented database designed for flexibility, scalability, and ease of development.
- **Overview:** It stores data in flexible, JSON-like documents, known as BSON (Binary JSON)



Key Features

NoSQL: Not restricted to the traditional relational database model.

Schema-less: Allows dynamic and flexible data schema.

Horizontal Scalability: Easily scales horizontally to handle large datasets.

Rich Query Language: Supports a variety of queries and indexing options.

High Performance: Optimized for read and write-intensive workloads.



Document-Oriented Data Model

Overview: MongoDB uses a document-oriented data model, where data is stored in BSON documents.

Documents: JSON-like structures with key-value pairs.

Collections: A group of MongoDB documents, equivalent to a table in a relational database.

Definition: BSON (Binary JSON) is a binary-encoded serialization of JSON-like documents.

Structure: Similar to JSON but includes additional data types like Date, Binary, and ObjectId.

Efficient Storage: Compact and efficient for storage and transmission.



MongoDB Data type

Data Types	Description
String	String is the most commonly used datatype. It is used to store data. A string must be UTF 8 valid in mongodb.
Integer	Integer is used to store the numeric value. It can be 32 bit or 64 bit depending on the server you are using.
Boolean	This datatype is used to store boolean values. It just shows YES/NO values.
Double	Double datatype stores floating point values.
Min/Max Keys	This datatype compare a value against the lowest and highest bson elements.
Arrays	This datatype is used to store a list or multiple values into a single key.
Object	Object datatype is used for embedded documents.
Null	It is used to store null values.
Symbol	It is generally used for languages that use a specific type.
Date	This datatype stores the current date or time in unix time format. It makes you possible to specify your own date time by creating object of date and pass the value of date, month, and year into it.



Installation and configuration of MongoDB

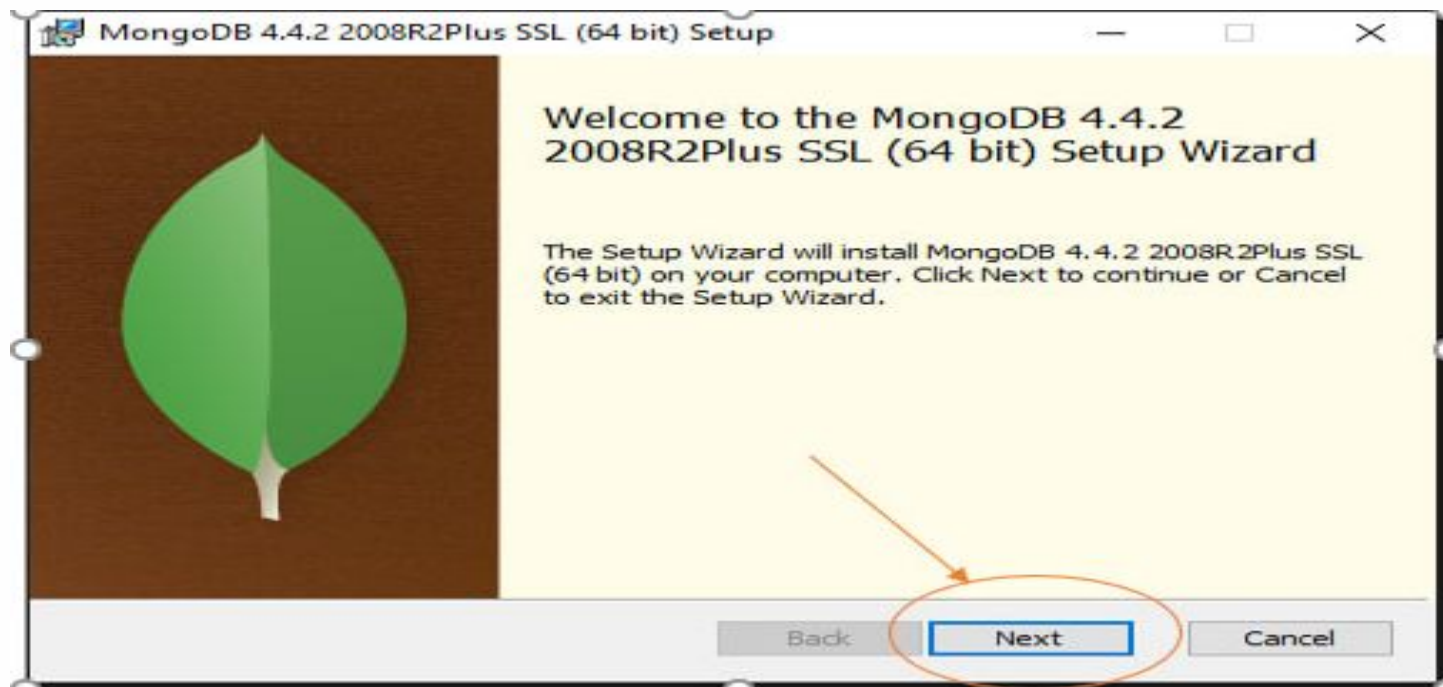
Step 1: Go to [MongoDB Download Center](#) to download MongoDB Community Server. Here, You can select any version, Windows, and package according to your requirement

A screenshot of the MongoDB website's download center. The top navigation bar includes links for 'mongoDB', 'Cloud', 'Software', 'Pricing', 'Learn', 'Solutions', and 'Docs', along with a search icon, 'Contact', 'Sign In', and a green 'Try Free' button. A dropdown menu is open, showing 'MongoDB Enterprise Server' and 'MongoDB Community Server'. The 'MongoDB Community Server' option is selected, displaying a description of the community version and a 'Available Downloads' section. This section has three dropdown menus: 'Version' set to '4.4.3 (current)', 'Platform' set to 'Windows', and 'Package' set to 'msi'. Below these is a green 'Download' button with a download icon, which is circled in orange with an arrow pointing to it from the left. To the right of the button is a 'Copy Link' option.



Contn...

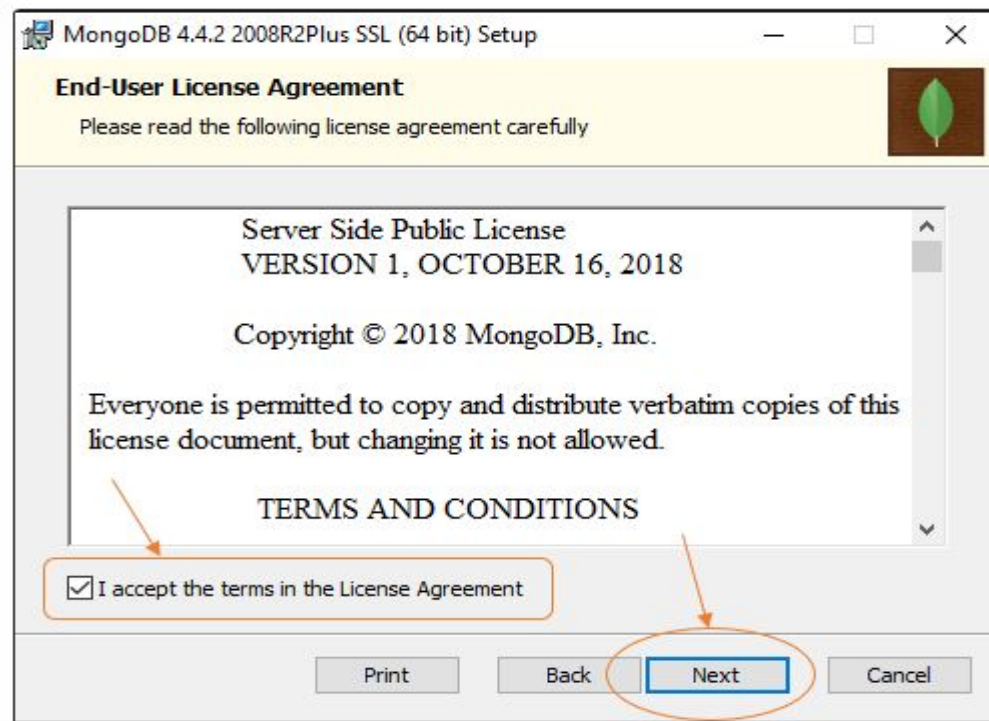
Step 2: When the download is complete open the msi file and click the *next button* in the startup screen:





Contn...

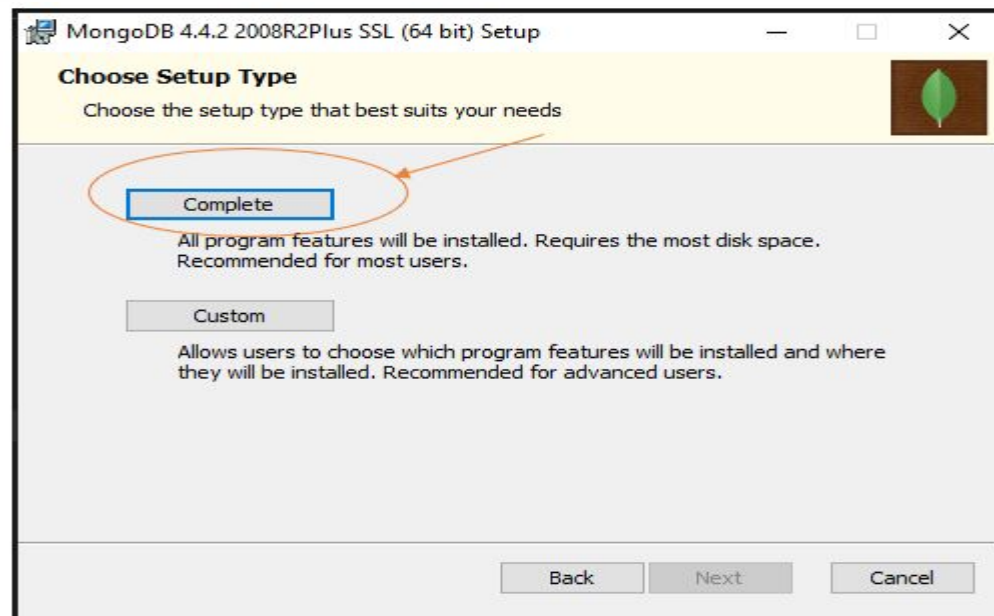
Step 3: Now accept the End-User License Agreement and click the next button





Contn...

Step 4: Now select the *complete option* to install all the program features. Here, if you can want to install only selected program features and want to select the location of the installation, then use the *Custom option*:





Contn...

Step 5: Select “Run service as Network Service user” and copy the path of the data directory. Click Next

MongoDB 4.4.2 2008R2Plus SSL (64 bit) Service Customization

Service Configuration
Specify optional settings to configure MongoDB as a service.

☒ Install MongoDB as a Service

☒ Run service as Network Service user

☐ Run service as a local or domain user:

Account Domain:

Account Name:

Account Password:

Service Name:

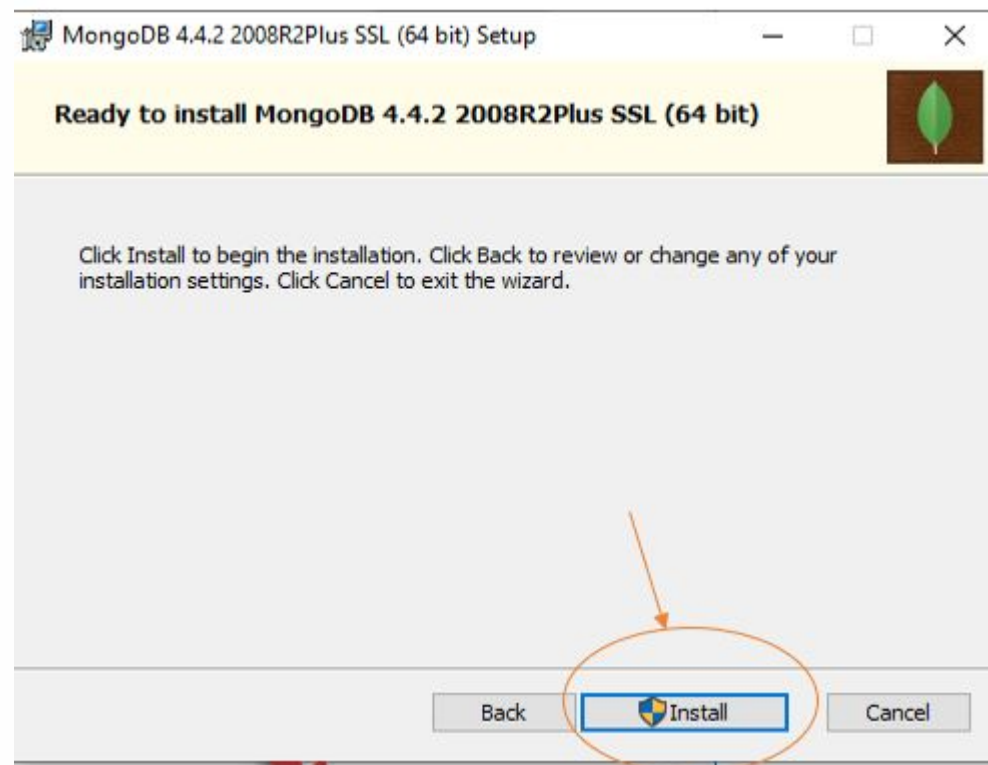
Data Directory:

Log Directory:

< Back Next > Cancel

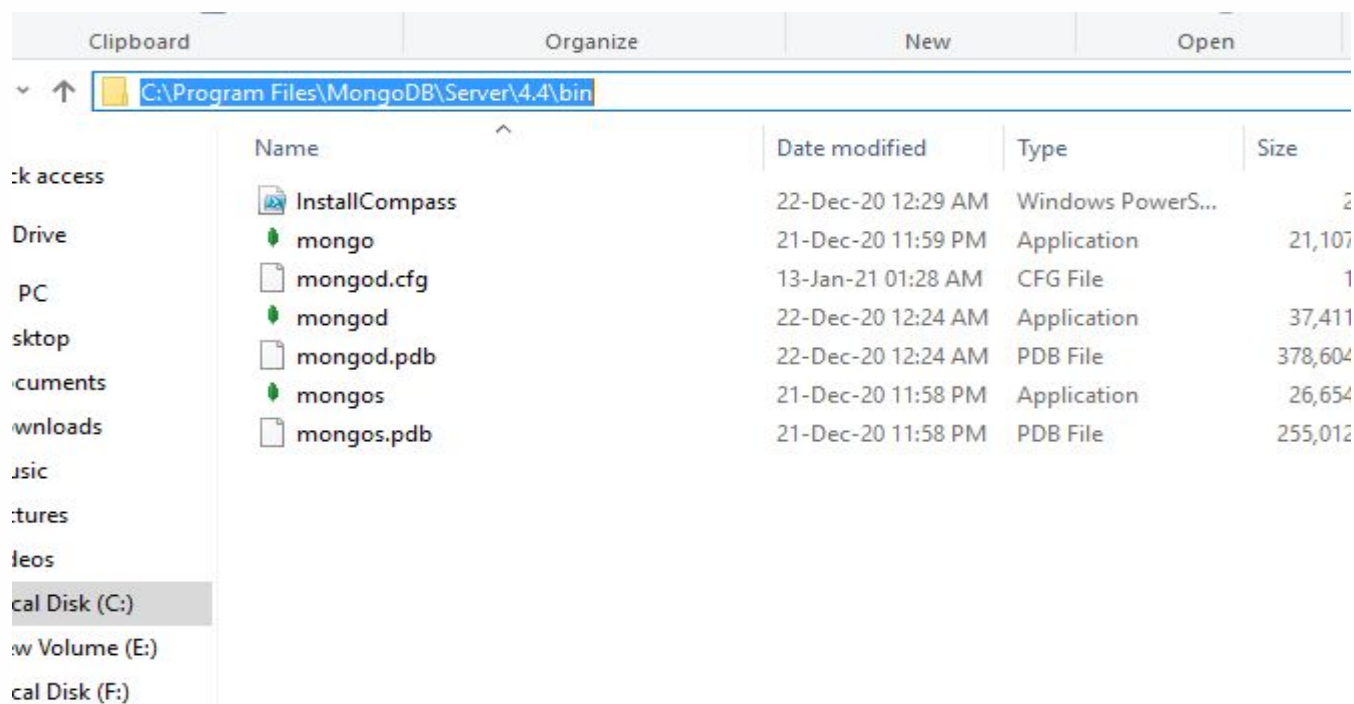
Contn...

Step 6: Click the *Install* button to start the installation process and then click Finish:



Contn...

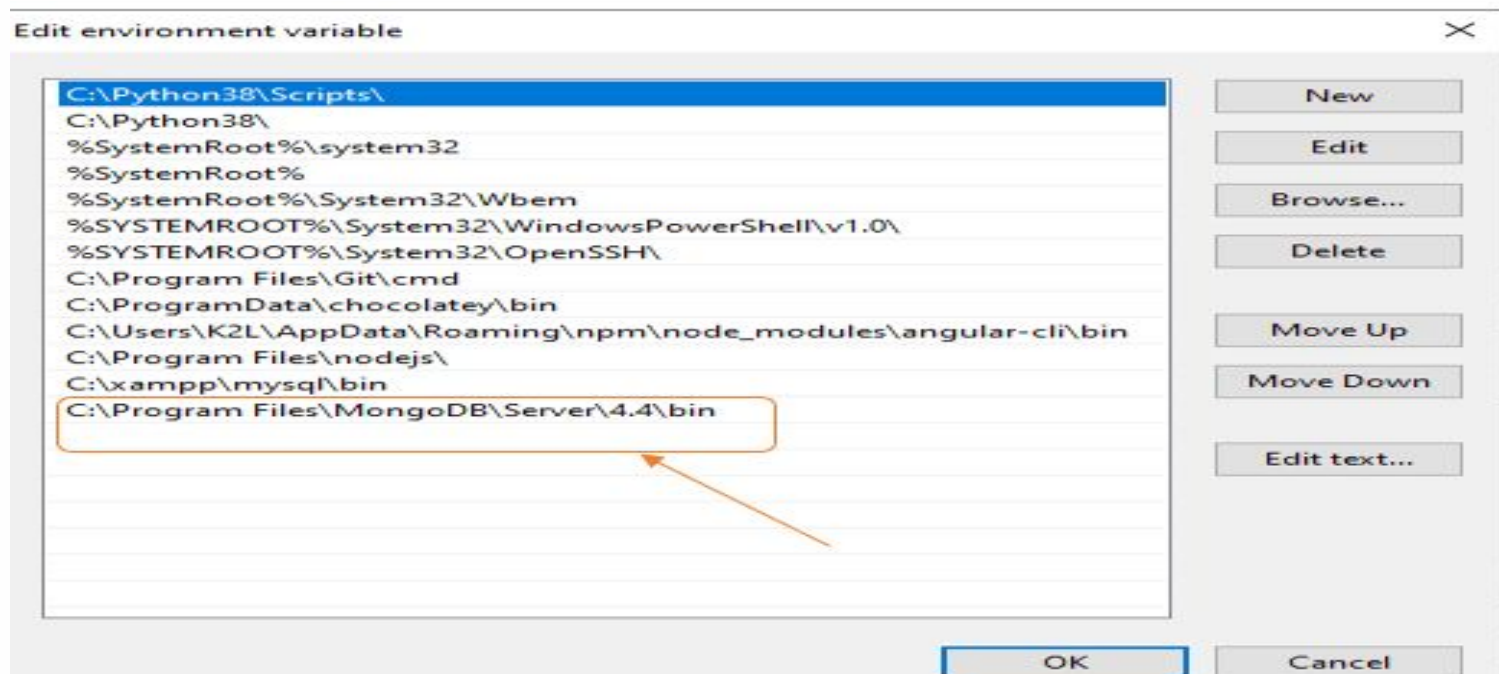
Step 7: Now we go to the location where MongoDB installed in step 5 in your system and copy the bin path:





Contn...

Step 8: Now, to create an environment variable open system properties << Environment Variable << System variable << path << Edit Environment variable and paste the copied link to your environment system and click Ok:





Contn...

Step 9: Now, Open C drive and create a folder named “data” inside this folder create another folder named “db”. After creating these folders. open the command prompt and run the following command:

Command: mongod

the MongoDB server(i.e., mongod) will run successfully.



Contn...

Step 10: Now we are going to connect our server (mongod) with the mongo shell. So, keep that mongod window and open a new command prompt window and write **mongo**. Now, our mongo shell will successfully connect to the mongod.

Important Point: Please do not close the mongod window if you close this window your server will stop working and it will not be able to connect with the mongo shell.

Now, you are ready to write queries in the mongo Shell



CRUD in MongoDB

CREATE, READ, UPDATE, and DELETE actions are referred to as CRUD operations in MongoDB. These are the basic operations used to alter data in databases. In MongoDB, the term "CRUD operations" refers to the standard set of database operations used to work with collections of data.

Create: To add new documents to a collection, use the Create operation.

Read: Data from a collection is retrieved using the Read operation.

Update: The Update operation is used to edit existing documents in a collection.

Delete: A collection of documents can be deleted using the Delete procedure.

Contn...

To create a new database, you can simply run any command against a non-existing database, and MongoDB will automatically create it for you.

If Database already exist then it use same existing database

Let us create a database name test

use test; (execute this command to create database)

Contn...

Creating a New Collection:

To create a new collection, you can use the "createCollection" method.

Syntax: `db.createCollection(name, options)`

eg:

```
db.createCollection("dummy")
```

Contn...

Create Operation

There are two ways to create new documents to a collection in MongoDB:

1. **insertOne():**

Adding a single document to a collection is done using this method. A document to be added is the only argument it accepts, and it returns a result object with details about the insertion.

Syntax: db.collectionName.insertOne()

eg:

```
db.dummy.insertOne({  
  name: "RAM",  
  age: 25,  
  ...  
});
```




Contn...

2. insertMany()

This method is used to insert multiple documents into a collection at once. It takes an array of documents as its argument and returns a result object that contains information about the insertion.

Syntax: db.collectionName.insertMany();

eg:

```
db.users.insertMany([
  {
    name: "Ram",
    age: 27,
  },
  {
    name: "Sham",
    age: 30,
  },
]);
```

Contn...

Read Operations

In MongoDB, read operations are used to retrieve data from the database.

1. find()

The find() method is used to retrieve data from a collection. It returns a cursor that can be iterated to access all the documents that match the specified query criteria.

Syntax: `db.collectionName.find(query, projection)`

Eg:

1. `db.dummy.find()`
2. `db.dummy.find({ age: { $gt: 29 } }, { name: 1, age: 1 })`

This command will return all the documents in the “dummy” collection where the age is greater than 29, and only return the “name” and “age” fields.

Contn...

2. findOne()

The findOne() method returns a single document object, or null if no document is found. You can pass a query object to this method to filter the results.

Syntax: `db.collectionName.findOne()`

Eg:

```
db.dummy.findOne({ name: "Ram" })
```

Contn...

1. **limit()** Method:

Limits the number of documents returned by a MongoDB query.

query: `db.students.find().limit(5);`

Limits the result set to the first 5 documents returned by the query.

2. **skip()** Method:

Skips a specified number of documents in a MongoDB query result set.

query: `db.students.find().skip(10);`

Skips the first 10 documents in the result set, returning the subsequent documents.



Contn...

Update Operations

In MongoDB, the "update" operation is used to modify existing documents in a collection.

1. updateOne()

The updateOne() method is used to update a single document that matches a specified filter.

Syntax: `db.collectionName.updateOne(filter, update, options)`

Eg:

```
db.dummy.updateOne({ name: "Ram" }, { $set: { age: "30" } })
```



Contn...

Following are a few of the many available operations:

\$set: Sets the value of a field in a document. If the field does not exist, the set will create it.

\$unset: Removes a field from a document.

\$inc: Increments the value of a field in a document by a specified amount.

\$push: Adds an element to the end of an array field in a document. If the field does not exist, push will create it as an array with the specified element.

\$pull: Removes all occurrences of a specified value from an array field in a document.

Contn...

2. updateMany

The updateMany() method is used to update multiple documents that match a specified filter.

Syntax:

db.collectionName.updateMany(filter, update, options)

Eg:

db.dummy.updateMany({ age: { \$lt: 30 } }, { \$set: { age: 28} })

This command will update the status of all documents in the “dummy” collection where the age is less than 30 to age 28.

Contn...

3. `replaceOne()`:

Replaces a single document with a new document.

```
db.students.replaceOne(  
  { name: "Bob" },  
  { name: "Bob", age: 29, grade: "A-" }  
);
```

Contn...

Delete Operations

In MongoDB, the "delete" operation is used to remove documents from a collection.

There are several ways to perform a delete operation, including the following:

1. deleteOne()

The deleteOne() method is used to remove a single document that matches a specified filter.

Syntax: db.collectionName.deleteOne(filter, options)

Eg:

```
db.dummy.deleteOne({ name: "ram" })
```

This command will remove the first document in the "dummy" collection where the name is "ram"

Contn...

2. deleteMany()

The deleteMany() method is used to remove multiple documents that match a specified filter.

Syntax:

db.collectionName.deleteMany(filter, options)

Eg:

db.dummy.deleteMany({ age: { \$lt: 30 } })

This command will remove all documents in the "users" collection where the age is less than 30



Contn...

3. drop()

The drop() method is used to remove an entire collection.

Syntax:

db.collectionName.drop()

Eg:

db.dummy.drop()

This command will remove the dummy collection.



Operators

\$eq

\$lt

\$gt

\$gte

\$lte

\$in: match value in array { **age:** { **\$in:** [25, 30, 35] } }

Operators

\$and db.users.find({\$and:[{age:{<25}},{name{\$eq:"Aman"}}]})

\$or

\$pull { \$pull: { tags: "mongodb" } }

\$set

\$unset { \$unset: { address: "" } }



Operators

\$inc

\$push

\$pop { \$pop: { tags: 1 } } — Removes the last element of the tags array.

\$rename



Indexing and querying in MongoDB

Indexes support the efficient resolution of queries. Without indexes, MongoDB must scan every document of a collection to select those documents that match the query statement. This scan is highly inefficient and require MongoDB to process a large volume of data.

Indexes are special data structures, that store a small portion of the data set in an easy-to-traverse form. The index stores the value of a specific field or set of fields, ordered by the value of the field as specified in the index.

To create an index, you need to use `createIndex()` method of MongoDB.

Syntax: **`db.collectionName.createIndex({KEY:1})`**

Contn...

Here key is the name of the field on which you want to create index and 1 is for ascending order. To create index in descending order you need to use -1.

Eg; db.dummy.createIndex({"title":1})

In **createIndex()** method you can pass multiple fields, to create index on multiple fields.

Eg; db.dummy.createIndex({"title":1, "description":1})

Contn...

The dropIndex() method

You can drop a particular index using the dropIndex() method of MongoDB.

Syntax: **Eg; db.collectionName.dropIndex({"KEY":1})**

Eg: Eg; db.dummy.createIndex({"title":1})



Schema design and data modeling

MongoDB Schema Design

Now, MongoDB schema design works a lot differently than relational schema design. With MongoDB schema design, there is:

- No formal process
- No algorithms
- No rules

Schema design and data modeling

When you are designing your MongoDB schema design, the only thing that matters is that you design a schema that will work well for your application. Two different apps that use the same exact data might have very different schemas if the applications are used differently. When designing a schema, we want to take into consideration the following:

Store the data

Provide good query performance

Require reasonable amount of hardware



Schema design and data modeling

```
{
  _id: POST_ID
  title: TITLE_OF_POST,
  description: POST_DESCRIPTION,
  by: POST_BY,
  url: URL_OF_POST,
  tags: [TAG1, TAG2, TAG3],
  likes: TOTAL_LIKES,
  comments: [
    {
      user: 'COMMENT_BY',
      message: TEXT,
      datecreated: DATE_TIME,
      like: LIKES
    },
    {
      user: 'COMMENT_BY',
      message: TEST,
      dateCreated: DATE_TIME,
      like: LIKES
    }
  ]
}
```



Schema design and data modeling

You can see that instead of splitting our data up into separate collections or documents, we take advantage of MongoDB's document based design to embed data into arrays and objects within the User object. Now we can make one simple query to pull all that data together for our application.

Schema design and data modeling

MongoDB schema design actually comes down to only two choices for every piece of data. You can either embed that data directly or reference another piece of data using the \$lookup operator (similar to a JOIN). Let's look at the pros and cons of using each option in your schema.

1.Embedding

You can retrieve all relevant information in a single query.

Avoid implementing joins in application code or using \$lookup.

Update related information as a single atomic operation.



Schema design and data modeling

2. Referencing

The other option for designing our schema is referencing another document using a document's unique ID and connecting them together using the \$lookup operator. Referencing works similarly as the JOIN operator in an SQL query. It allows us to split up data to make more efficient and scalable queries, yet maintain relationships between data.

By splitting up data, you will have smaller documents.

Infrequently accessed information not needed on every query.

Reduce the amount of duplication of data. However, it's important to note that data duplication should not be avoided if it results in a better schema.

Schema design and data modeling

Type of Relationships

1. One-to-one

Using embedded documents we can create one-to-one relationships between the data so, that we can easily retrieve data using few read operations

```
// Student document {   StudentName: Ishan,  
                        StudentId: 123_2022,  
                        Branch:CSE }
```

```
// Address document  
{   StudentName: Ishan,  
    PremanentAddress: XXXXXXXX,  
    City: Jaipur,  
    PinCode:302022 }
```



Schema design and data modeling

If the address data is frequently used, the user creates a query using Student Name to retrieve the data from the address document. However, because two documents contain the same field (i.e., StudentName), the user must write a few more queries to retrieve the required information. This data retrieval procedure is time-consuming. As a result, the address document was embedded in the student document.

```
{  
  StudentName: Ishan,  
  StudentId: 123_2020,  
  Branch:CSE  
  PermanentAddress:{  
    PremanentAddress:XXXXXXX,  
    City: Jaipur,  
    PinCode:302022  } }
```



Schema design and data modeling

2. One-to-Many

We can create one-to-many relationships between data using embedded documents, allowing us to retrieve data quickly with few read operations. With the help of an example, we will now discuss the one-to-many relationship with embedded documents.

Schema design and data modeling

2. One-to-Many

// Student document

```
{  StudentName: Ishan,  
  StudentId: 123_2022,  
  Branch:CSE  
}
```

// Permanent Address document

```
{  StudentName: Ishan,  
  PermanentAddress: XXXXXXXX,  
  City: Jaipur,  
  PinCode:302022 }
```

// Current Address document

```
{  
  StudentName: Ishan,  
  CurrentAddress: XXXXXXXX,  
  City: Kota,  
  PinCode:324001 }
```

Schema design and data modeling

Instead of writing three documents, we can now combine them into one.

// Student document

```
{  StudentName: ishan,
  StudentId: k_hut_2022,
  Branch:CSE
  Address: [
    {
      StudentName: Ishan,
      PermanentAddress: XXXXXXXX,
      City: Jaipur,
      PinCode:302022
    },
    {
      StudentName: Ishan,
      CurrentAddress: XXXXXXXX,
      City: kota,
      PinCode:324001    }  ] }
```




Schema design and data modeling

One-to-Many relationships with the document reference

The document reference model can also be used to create a one-to-many relationship. We keep the documents separate in this model, but one document contains the references to the others.

A Many-to-Many relationship

Many to Many relationships are a type of mongodb relationship in which any two entities within a document can have multiple relationships.

In this relationship, we can consider a case of Online courses website where there are many courses and also many users. Each course is purchased by many users and each user may purchase many courses.

MongoDB Query Operators

Comparison

The following operators can be used in queries to compare values:

\$eq	Matches values that are equal to the given value.
\$gt	Matches if values are greater than the given value.
\$lt	Matches if values are less than the given value.
\$gte	Matches if values are greater or equal to the given value.
\$lte	Matches if values are less or equal to the given value.
\$in	Matches any of the values in an array.
\$ne	Matches values that are not equal to the given value.
\$nin	Matches none of the values specified in an array

MongoDB Query Operators

Comparison

The following operators can be used in queries to compare values:

\$eq	Matches values that are equal to the given value.
\$gt	Matches if values are greater than the given value.
\$lt	Matches if values are less than the given value.
\$gte	Matches if values are greater or equal to the given value.
\$lte	Matches if values are less or equal to the given value.
\$in	Matches any of the values in an array.
\$ne	Matches values that are not equal to the given value.
\$nin	Matches none of the values specified in an array



MongoDB Query Operators

Logical

The following operators can logically compare multiple queries.

\$and	Joins two or more queries with a logical AND and returns the documents that match all the conditions.
\$or	Join two or more queries with a logical OR and return the documents that match either query.
\$nor	The opposite of the OR operator. The logical NOR operator will join two or more queries and return documents that do not match the given query conditions.
\$not	Returns the documents that do not match the given query expression.

MongoDB Aggrigation

In MongoDB, aggregation operations process the data records/documents and return computed results. It collects values from various documents and groups them together and then performs different types of operations on that grouped data like sum, average, minimum, maximum, etc to return a computed result.

MongoDB provides three ways to perform aggregation

Aggregation pipeline

Map-reduce function

Single-purpose aggregation



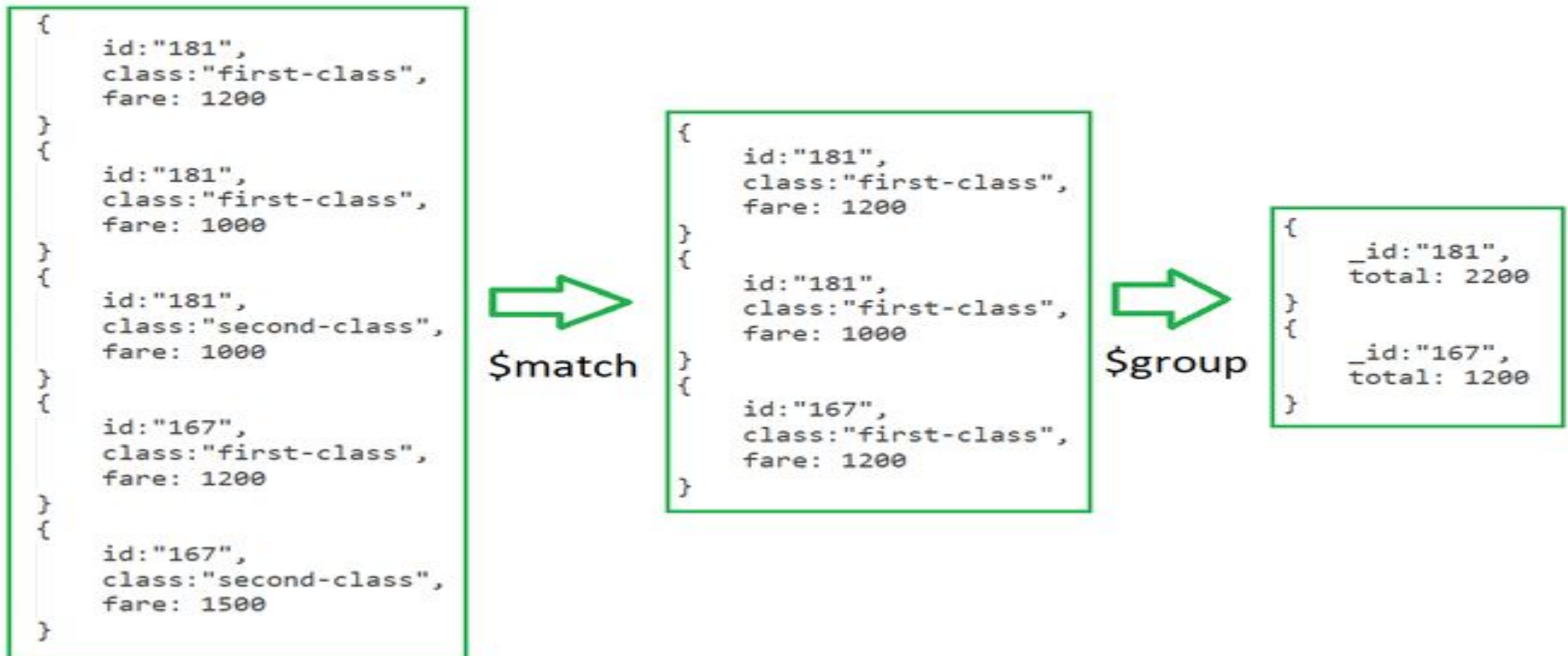
Aggrigation Pipeline

In MongoDB, the aggregation pipeline consists of stages and each stage transforms the document. Or in other words, the aggregation pipeline is a multi-stage pipeline, so in each state, the documents taken as input and produce the resultant set of documents now in the next stage the resultant documents taken as input and produce output, this process is going on till the last stage. The basic pipeline stages provide filters that will perform like queries and the document transformation modifies the resultant document and the other pipeline provides tools for grouping and sorting documents.



Aggrigation Pipeline

```
db.train.aggregate( [
  {$match:{class:"first-class"}},
  {$group:{_id:"id",total:{$sum:"$fare"}}} ] pipeline stages
)
```



Aggrigation Pipeline

In the previous example of a collection of train fares in the first stage. Here, the \$match stage filters the documents by the value in class field i.e. class: “first-class” and passes the document to the second stage.

In the Second Stage, the \$group stage groups the documents by the id field to calculate the sum of fare for each unique id.



Aggrigation Pipeline

Stages: Each stage starts from stage operators which are:

1. **\$match:** It is used for filtering the documents can reduce the amount of documents that are given as input to the next stage.
2. **\$project:** It is used to select some specific fields from a collection.
3. **\$group:** It is used to group documents based on some value.
4. **\$sort:** It is used to sort the document that is rearranging them

Aggrigation Pipeline

5. \$skip: It is used to skip n number of documents and passes the remaining documents
6. \$limit: It is used to pass first n number of documents thus limiting them.
7. \$unwind: It is used to unwind documents that are using arrays i.e. it deconstructs an array field in the documents to return documents for each element.
8. \$out: It is used to write resulting documents to a new collection.

Aggrigation Pipeline

Expressions: It refers to the name of the field in input documents for e.g. { \$group : { _id : “\$id“, total:{ \$sum:”\$fare“} } } here \$id and \$fare are expressions.

Accumulators: These are basically used in the group stage

1. sum: It sums numeric values for the documents in each group
2. count: It counts total numbers of documents

Aggrigation Pipeline

- 3. avg: It calculates the average of all given values from all documents
- 4. min: It gets the minimum value from all the documents
- 5. max: It gets the maximum value from all the documents
- 6. first: It gets the first document from the grouping
- 7. last: It gets the last document from the grouping



Aggrigation Pipeline

```
db.dummy.aggregate([  
  // Stage 1: Only find documents that have more than 1 like  
  {  
    $match: { likes: { $gt: 1 } }  
  },  
  // Stage 2: Group documents by category and sum each categories likes  
  {  
    $group: { _id: "$category", totalLikes: { $sum: "$likes" } }  
  }  
])
```

Aggrigation Pipeline

```
db.restaurants.aggregate([
  {
    $project: {
      "name": 1,
      "cuisine": 1,
      "address": 1
    }
  },
  {
    $limit: 5
  }
])
```

Aggrigation Pipeline

```
db.listingsAndReviews.aggregate([
  {
    $sort: { "accommodates": -1 }
  },
  {
    $project: {
      "name": 1,
      "accommodates": 1
    }
  },
  {
    $limit: 5
  }
])
```


Aggrigation Pipeline

```
db.listingsAndReviews.aggregate([  
  { $match : { property_type : "House" } },  
  { $limit: 2 },  
  { $project: {  
    "name": 1,  
    "bedrooms": 1,  
    "price": 1  
  }}  
])
```



Aggrigation Pipeline

\$addField: This will return the documents along with a new field, avgGrade, which will contain the average of each restaurants grades.score.

```
db.restaurants.aggregate([
  {
    $addFields: {
      avgGrade: { $avg: "$grades.score" }
    }
  },
  {
    $project: {
      "name": 1,
      "avgGrade": 1
    }
  }
])
```



Aggrigation Pipeline

\$lookup: This aggregation stage performs a left outer join to a collection in the same database.

from: The collection to use for lookup in the same database

localField: The field in the primary collection that can be used as a unique identifier in the from collection.

foreignField: The field in the from collection that can be used as a unique identifier in the primary collection.

as: The name of the new field that will contain the matching documents from the from collection.

Aggrigation Pipeline

```
db.comments.aggregate([
  {
    $lookup: {
      from: "movies",
      localField: "movie_id",
      foreignField: "_id",
      as: "movie_details",
    },
  },
  {
    $limit: 1
  }
])
```

Aggrigation Pipeline

1. **findOneAndUpdate():**

Finds a single document that matches a specified query criteria, updates it, and returns the original or modified document.

example:

```
const result = db.students.findOneAndUpdate(  
  { name: "Alice" }, // Query criteria  
  { $set: { grade: "A+" } }, // Update operation  
  { returnDocument: "after" } // Options to return the modified document  
);  
  
printjson(result.value);
```

Aggrigation Pipeline

1. **findOneAndUpdate():**

Finds a single document that matches a specified query criteria, updates it, and returns the original or modified document.

example:

```
const result = db.students.findOneAndUpdate(  
  { name: "Alice" }, // Query criteria  
  { $set: { grade: "A+" } }, // Update operation  
  { returnDocument: "after" } // Options to return the modified document  
);  
  
printjson(result.value);
```

Aggrigation Pipeline

2. **findOneAndDelete():**

Finds a single document that matches a specified query criteria, deletes it, and returns the original or deleted document.

example

```
const result = db.students.findOneAndDelete(  
  { name: "Bob" } // Query criteria  
);
```

Aggrigation Pipeline

3. **findOneAndReplace():**

Finds a single document that matches a specified query criteria, replaces it with a new document, and returns the original or replaced document.

```
const result = db.students.findOneAndReplace(  
  { name: "Charlie" }, // Query criteria  
  { name: "Charlie", age: 24, grade: "A-" }, // Replacement document  
  { returnDocument: "before" } // Options to return the original document  
);
```


x DIGITAL LEARNING CONTENT

o



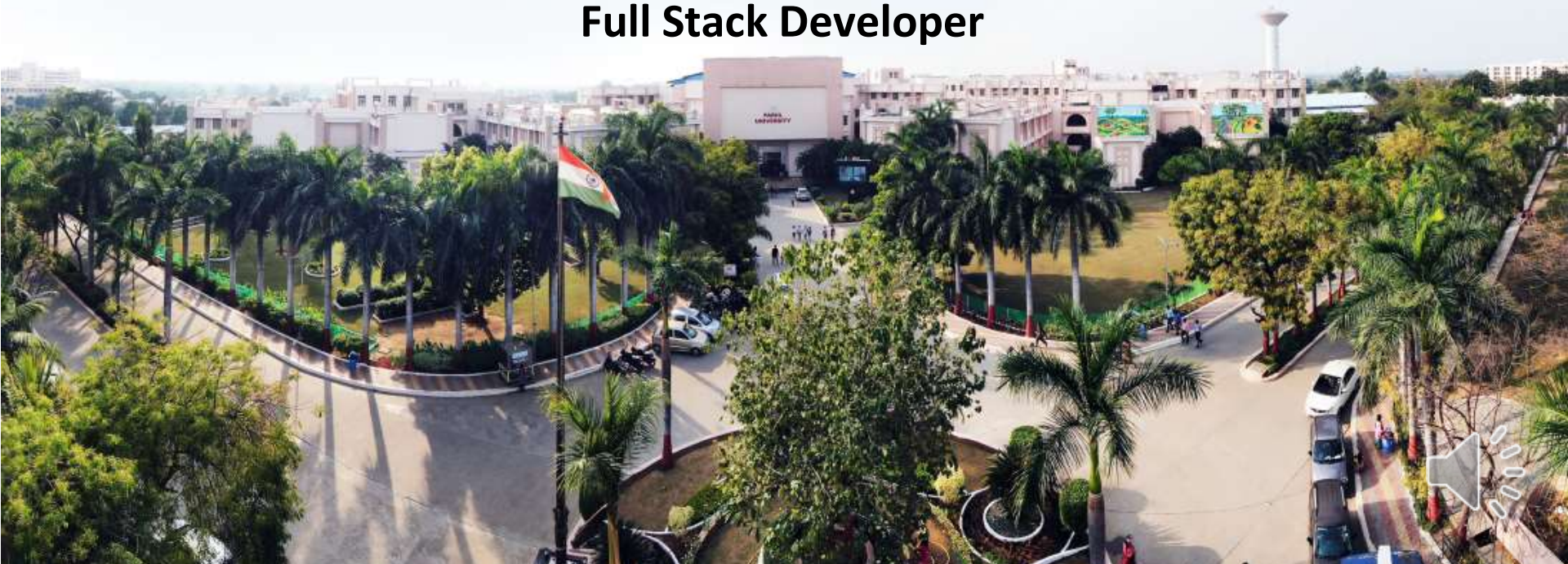
Parul[®] University



www.paruluniversity.ac.in

MEAN Full Stack

Amir Hussain
Cyber security trainer
&
Full Stack Developer





CHAPTER-3rd

Introduction



Node.JS

Node JS is an open-source and cross-platform runtime environment built on Chrome's V8 JavaScript engine for executing JavaScript code outside of a browser. It provides an event-driven, non-blocking (asynchronous) I/O and cross-platform runtime environment for building highly scalable server-side applications using JavaScript.

Node.js also provides a rich library of various JavaScript modules which simplifies the development of web applications using Node.js to a great extent.

Node.js = Runtime Environment + JavaScript Library

Node.JS

Node.js is an open-source and cross-platform runtime environment for executing JavaScript code outside a browser.

NodeJS is not a framework and it's not a programming language.

Node.js is used to build back-end services like **APIs** like Web App or Mobile App.

It's used in production by large companies such as Paypal, Uber, Netflix, Walmart, and so on.



Why do we use Node.JS

There are many reasons for which we prefer using NodeJs for the server side of our application, some of them are discussed in the following:

- NodeJs is built on Google Chrome's V8 engine, and for this reason its execution time is very fast and it runs very quickly.
- There are more than 50,000 bundles available in the Node Package Manager and for that reason developers can import any of the packages any time according to their needed functionality for which a lot of time is saved.
- As NodeJs do not need to wait for an API to return data , so for building real time and data intensive web applications, it is very useful. It is totally asynchronous in nature that means it is totally non-blocking.



Why do we use Node.JS

- The loading time for an audio or video is reduced by NodeJs because there is better synchronization of the code between the client and server for having the same code base.
- As NodeJs is open-source and it is nothing but a JavaScript library, so for the developers who are already used to JavaScript, for them starting developing their projects with NodeJs is very easy.



Node.JS

FEATURE OF NODE.JS

1. **Asynchronous and Event Driven** – All APIs of Node.js library are asynchronous, that is, non-blocking. It essentially means a Node.js based server never waits for an API to return data. The server moves to the next API after calling it and a notification mechanism of Events of Node.js helps the server to get a response from the previous API call
2. **Very Fast** – Being built on Google Chrome's V8 JavaScript Engine, Node.js library is very fast in code execution.



Node.JS

FEATURE OF NODE.JS

3. **Single Threaded but Highly Scalable** – Node.js uses a single threaded model with event looping. Event mechanism helps the server to respond in a non-blocking way and makes the server highly scalable as opposed to traditional servers which create limited threads to handle requests. Node.js uses a single threaded program and the same program can provide service to a much larger number of requests than traditional servers like Apache HTTP Server.
4. **No Buffering** – Node.js applications never buffer any data. These applications simply output the data in chunks.



Node.JS

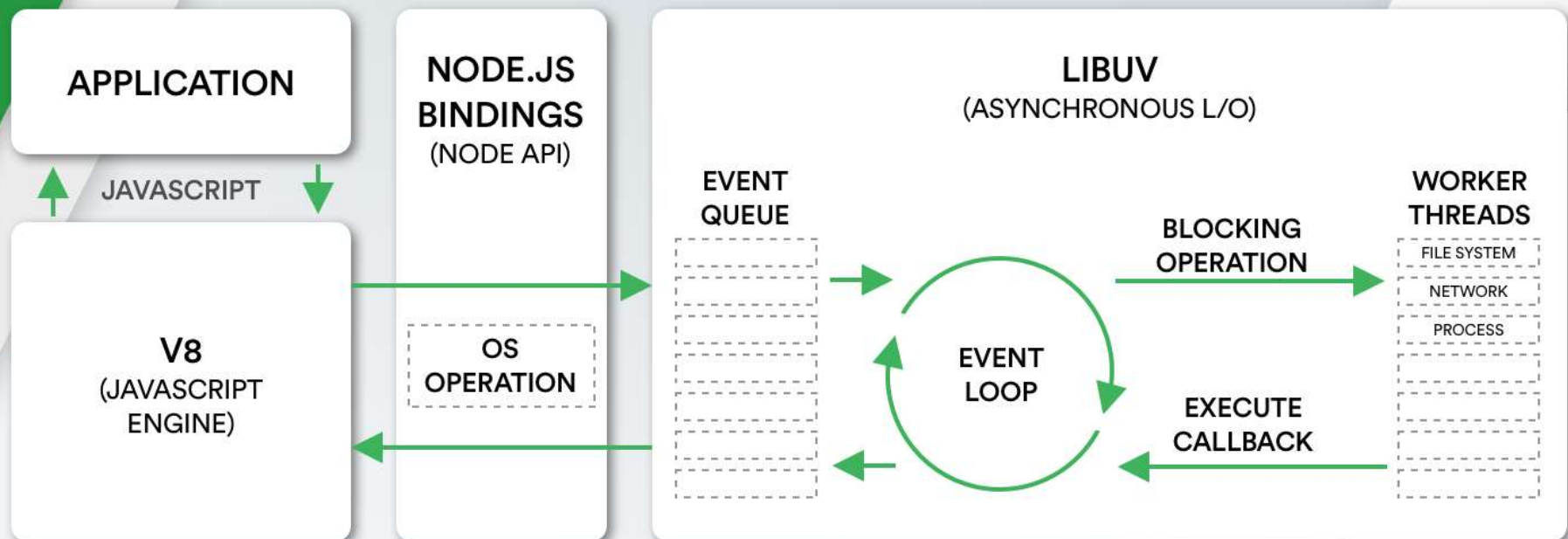
FEATURE OF NODE.JS

5. **Real time web apps:** Today the web has become much more about interaction. Users want to interact with each other in real-time. Chat, gaming, constant social media updates, collaboration tools, eCommerce websites, real-time tracking apps, marketplace- each of these features requires real-time communication between users, clients, and servers across the web.
6. **Compatibility on the cross platforms:** Different types of systems like Windows, UNIX, LINUX, MacOS and other mobile devices can use NodeJs. For generating a self-sufficient execution, it can be paired with any appropriate package.

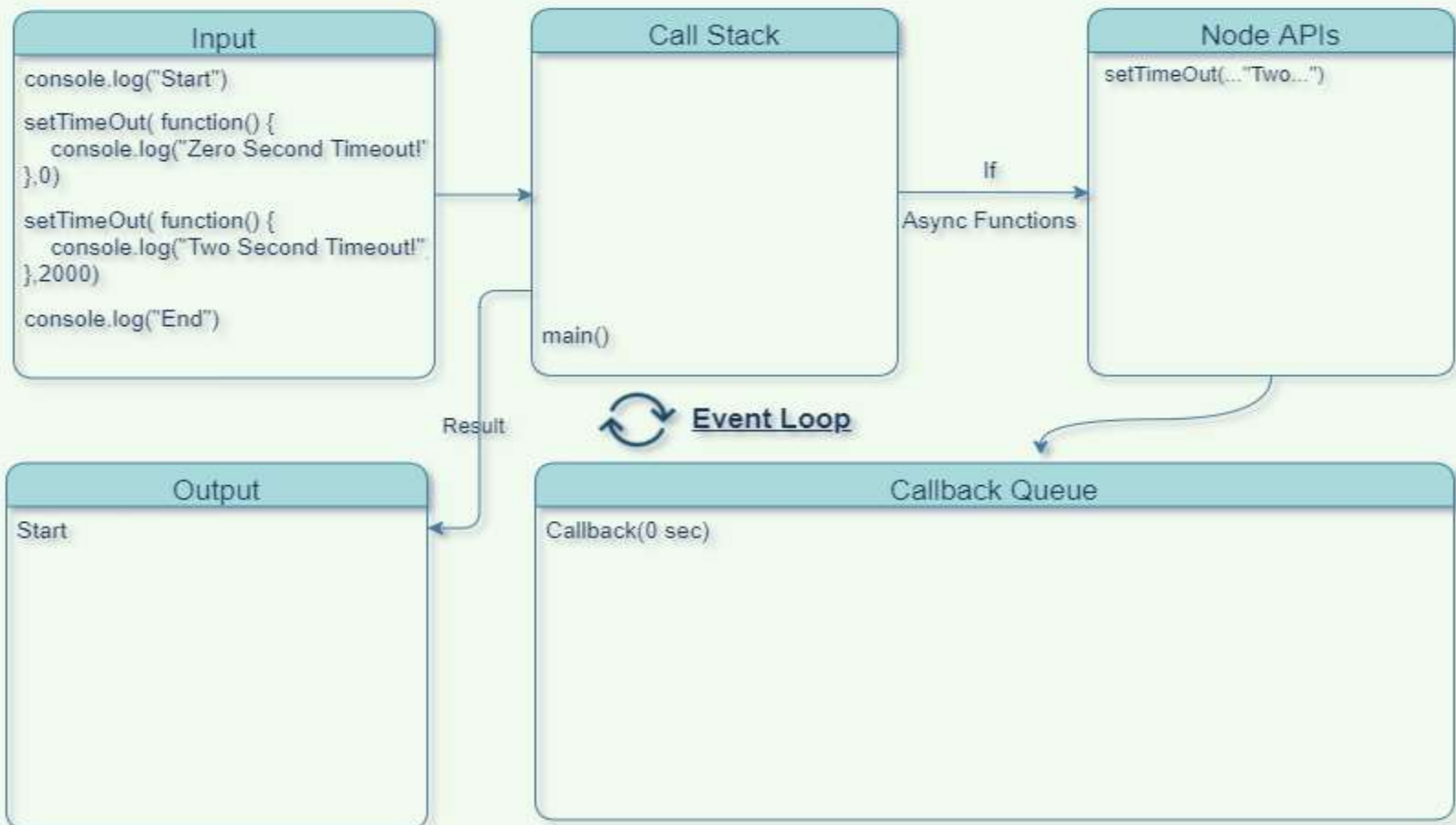


Node.JS Architecture

Node.js Architecture



Working of event loop





Working of event loop

The event loop in Node.js is a fundamental concept that allows it to handle asynchronous operations efficiently. Here's a concise explanation for your PowerPoint presentation

In essence, the event loop is the mechanism that allows Node.js to handle many operations concurrently, making it well-suited for scalable and high-performance applications.



Creating Node.js Application

Step 1 - Import Required Module

We use the **require** directive to load the http module and store the returned HTTP instance into an http variable as follows

```
var http= require("http");
```

Step 2 - Create Server

We use the created http instance and call **http.createServer()** method to create a server instance and then we bind it at port 8081 using the **listen** method associated with the server instance. Pass it a function with parameters request and response.

Creating Node.js Application

Step 3 - Testing Request & Response

Let's put step 1 and 2 together in a file called **main.js** and start our HTTP server as shown below –

```
const http = require('http');
const hostname = '127.0.0.1';
const port = 3000;
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Server running\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Creating Node.js Application

Step 3 - Testing Request & Response

Let's put step 1 and 2 together in a file called **main.js** and start our HTTP server as shown below –

```
const http = require('http');
const hostname = '127.0.0.1';
const port = 3000;
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Server running\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```


NPM

NPM is the default package manager for Node.js, a runtime environment for executing JavaScript code server-side. NPM allows developers to discover, install, and manage third-party packages (libraries, frameworks, tools, etc.) for their Node.js applications. It simplifies the process of integrating external modules and tools into a Node.js project, making it a fundamental tool for Node.js developers.

Key Features of NPM

1. Developers can use NPM to install packages from the NPM registry.

For example:

```
npm install package-name
```

NPM

2. Dependency Management:

NPM automatically manages dependencies, ensuring that the required libraries and tools are installed and accessible for a given project.

3. Version Control:

NPM allows developers to specify version ranges or exact versions of packages to ensure consistency and compatibility within a project.

4. Scripting:

NPM provides a scripting mechanism through the "scripts" field in the package.json file. Developers can define custom scripts, such as running tests or starting the application, and execute them with simple commands.



NPM

5. Global and Local Installations:

Packages can be installed globally to make them available system-wide or locally within a specific project.

1. To install a package locally and add it to the package.json file.

`npm install package-name --save`

2. Global Installations:

For packages needed system-wide:

`npm install -g package-name`

.

NodeJs Modules

In Node.js, a module is a reusable block of code that encapsulates related functionality and can be used to organize and structure a Node.js application. Modules in Node.js promote modular programming, making it easier to manage and maintain large codebases by breaking them down into smaller, more manageable pieces.

Types of Modules in Node.js:

1. Core Modules:

Core modules are built-in modules that come with the Node.js installation. They provide essential functionality for common tasks.

Example: fs (file system), http (HTTP server/client), path (path manipulation), etc.

Core modules are loaded using require without specifying a path, e.g.,
`const fs = require('fs');`



NodeJs Modules

2. Local Modules:

Local modules are modules created by developers to organize and encapsulate specific functionality within their Node.js application. A local module is typically a separate JavaScript file that can be loaded into other files using `require`.

Example: Suppose you have a file named `myModule.js`:

```
const myFunction = () => {  
  console.log('This is a custom module function.');  
};
```

```
module.exports = myFunction;
```

NodeJs Modules

2. Local Modules:

You can use this module in another file like this:

```
const myModule = require('./myModule');  
myModule(); // This is a custom module function.
```

NodeJs Modules

3. Third-Party Modules:

Third-party modules are modules created by external developers and shared through the Node Package Manager (NPM).

Developers can use NPM to install third-party modules and include them in their projects.

Example: express for building web applications, lodash for utility functions, etc.

Installation: `npm install module-name`

Usage: `const moduleName = require('module-name');`

Express.JS

Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It is an open source framework developed and maintained by the Node.js foundation.

Express provides a minimal interface to build our applications. It provides us the tools that are required to build our app. It is flexible as there are numerous modules available on **npm**, which can be directly plugged into Express.

To install Express and add it to our package.json file, use the following command

npm install --save express

Express.JS

To start developing our first app using Express. Create a new file called **index.js** and type the following in it.

```
const express = require('express');  
const app = express();  
app.get('/', (req, res) => { res.send('Hello, World!'); });  
const port = 3000;  
app.listen(port, () => { console.log(`Server running at http://localhost:${port}`); });
```

Save the file, go to your terminal and start application it will show
“Hello, World!”

ExpressJS - HTTP Methods

S.No.	Method & Description
	GET
1	The GET method requests a representation of the specified resource. Requests using GET should only retrieve data and should have no other effect.
	POST
2	The POST method requests that the server accept the data enclosed in the request as a new object/entity of the resource identified by the URI.
	PUT
3	The PUT method requests that the server accept the data enclosed in the request as a modification to existing object identified by the URI. If it does not exist then the PUT method should create one.
	DELETE
4	The DELETE method requests that the server delete the specified resource.



Express Middleware

Middleware in Node.js, particularly in the context of web frameworks like Express, is a function that has access to the request, response, and the next middleware function in the application's request-response cycle. Middleware functions can perform various tasks, modify the request or response objects, end the request-response cycle, and call the next middleware in the stack

Lets discuss an example...

Express Middleware

```
const express = require('express');
const app = express();

//Middlewares
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Custom middleware to log incoming requests
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});

app.get('/', (req, res) => {
  res.send('Hello, World!');
});

const port = 3000;
app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```



ExpressJS - Form data

Forms are an integral part of the web. Almost every website we visit offers us forms that submit or fetch some information for us. To get started with forms, we will first install the *body-parser* (for parsing JSON and url-encoded data) and *multer* (for parsing multipart/form data) middleware.

To install the *body-parser* and *multer*, go to your terminal and use...

```
npm install --save body-parser
```

```
npm install --save multer
```

After importing the body parser and multer, we will use the **body-parser** for parsing json and x-www-form-urlencoded header requests, while we will use **multer** for parsing multipart/form-data.



ExpressJS Database

We keep receiving requests, but end up not storing them anywhere. We need a Database to store the data. For this, we will make use of the NoSQL database called **MongoDB**.

In order to use Mongo with Express, we need a client API for node. There are multiple options for us, but for this tutorial, we will stick to mongoose. Mongoose is used for **document Modeling** in Node for MongoDB. For document modeling, we create a **Model** (much like a **class** in document oriented programming), and then we produce **documents** using this Model (like we create **documents of a class** in OOP). All our processing will be done on these "documents", then finally, we will write these documents in our database.



ExpressJS Database

We keep receiving requests, but end up not storing them anywhere. We need a Database to store the data. For this, we will make use of the NoSQL database called **MongoDB**.

In order to use Mongo with Express, we need a client API for node. There are multiple options for us, but for this tutorial, we will stick to mongoose. Mongoose is used for **document Modeling** in Node for MongoDB. For document modeling, we create a **Model** (much like a **class** in document oriented programming), and then we produce **documents** using this Model (like we create **documents of a class** in OOP). All our processing will be done on these "documents", then finally, we will write these documents in our database.



ExpressJS Middleware

Express.js middleware is a crucial aspect of building web applications with the MEAN (MongoDB, Express.js, AngularJS, Node.js) stack. Middleware functions in Express.js play a pivotal role in handling requests, modifying the request and response objects, and enabling various features within your application.

Middleware functions in Express.js are functions that have access to the request object (req), the response object (res), and the next middleware function in the application's request-response cycle. Middleware can perform tasks, modify the request or response, end the request-response cycle, and call the next middleware in the stack.



ExpressJS Middleware

Middleware in Express are functions that come into play **after the server receives the request and before the response is sent to the client**. They are arranged in a chain and are called in sequence.

We can use middleware functions for different types of processing tasks required for fulfilling the request like database querying, making API calls, preparing the response, etc, and finally calling the next middleware function in the chain



ExpressJS Middleware

Middleware functions take three arguments: the request object (request), the response object (response), and optionally the next() middleware function:

```
const express = require('express');
const app = express();
function middlewareFunction(request, response, next){
  ...
  next()
}

app.use(middlewareFunction)
```

An exception to this rule is error handling middleware which takes an error object as the fourth parameter. We call `app.use()` to add a middleware function to our Express application.

Type of ExpressJS Middleware

Application-level Middleware:

Application-level middleware is bound to the app object and executes for every request to the application.

```
app.use(function(req, res, next) {  
  // middleware logic  
  next();  
});
```

Application-level middleware which runs for all routes in an app object

Type of ExpressJS Middleware

Router-level Middleware:

Router-level middleware is bound to an instance of `express.Router()` and executes for routes defined on that router

```
var router = express.Router();  
router.use(function(req, res, next) {  
  // middleware logic  
  next();  
});
```

Router level middleware which runs for all routes in a router object

Type of ExpressJS Middleware

Error-handling Middleware:

Error-handling middleware is defined with four parameters, and it gets invoked when an error occurs during the request-response cycle.

```
app.use(function(err, req, res, next) {  
  // error-handling logic  
  res.status(500).send('Something went wrong!');  
});
```

Error handling middleware for handling errors

Type of ExpressJS Middleware

Built-in Middleware:

Express provides several built-in middleware functions that can be easily integrated into your application

- **`express.json()`: Parses incoming requests with JSON payloads.**
- **`express.urlencoded()`: Parses incoming requests with URL-encoded payloads.**
- **`express.static()`: Serves static files**



Type of ExpressJS Middleware

Third-party Middleware:

- Developers can use third-party middleware to add additional functionality to their applications.
- Example (using **body-parser** for parsing JSON or **multer** file upload):

```
const bodyParser = require('body-parser');  
app.use(bodyParser.json());
```

- Third-party middleware maintained by the community



Example of ExpressJS Middleware

Authentication Middleware:

Verifies if the user is authenticated before allowing access to protected routes.

```
function authenticate(req, res, next) {  
  if (req.isAuthenticated()) {  
    return next();  
  }  
  res.redirect('/login');  
}  
app.get('/dashboard', authenticate, function(req, res) {  
  res.render('dashboard');  
});
```

Middleware functions are executed in the order they are declared in your application code.

To move to the next middleware in the stack, the `next()` function must be called.



ExpressJS Routing

Routing in Express involves defining routes for handling different HTTP requests and specifying the corresponding logic to execute when a request matches a particular route. Express uses a chain of middleware functions to handle requests, and each route can have its own set of middleware

Routing defines the way in which the client requests are handled by the application endpoints.

Implementation of routing in Node.js:

There are two ways to implement routing in node.js which are listed below:

- By Using Framework
- Without using Framework



ExpressJS Routing

Using Framework:

Node has many frameworks to help you to get your server up and running. The most popular is Express.js.

Routing with Express in Node:

Express.js has an “app” object corresponding to HTTP. We define the routes by using the methods of this “app” object. This app object specifies a callback function called when a request is received. We have different methods of in-app objects for different types of requests.

ExpressJS Routing

```
var express = require('express');  
var app = express();  
  
app.get('/hello', function(req, res){  
  res.send("Hello World!");  
});  
  
app.listen(3000)
```

ExpressJS Routing

1. Basic Routing:

Use `app.get()`, `app.post()`, `app.put()`, `app.delete()`, etc., to define routes based on HTTP methods

```
const express = require('express');  
const app = express();
```

```
app.get('/', (req, res) => {  
  res.send('Hello, this is the homepage!');  
});
```

```
app.post('/users', (req, res) => {  
  res.send('Creating a new user');  
});
```

// Additional HTTP methods can be used for different routes



ExpressJS Routing

2. Route Parameters:

Route parameters are placeholders for values that are part of the URL.

Example:

Access route parameters using req.params.

```
app.get('/users/:id', (req, res) => {  
  const userId = req.params.id;  
  res.send(`User ID: ${userId}`);  
});
```

ExpressJS Routing

3. Handling Multiple Route Parameters:

Example:

Define routes with multiple parameters

```
app.get('/users/:category/:id', (req, res) => {  
  const category = req.params.category;  
  const userId = req.params.id;  
  res.send(`Category: ${category}, User ID: ${userId}`);  
});
```

ExpressJS Routing

4. Route Middleware:

Middleware functions that have access to the request, response, and next function in the application's request-response cycle.

Example:

Implement middleware for route-specific tasks.

```
const logRequest = (req, res, next) => {  
  console.log(`Received request for ${req.url}`);  
  next();  
};
```

```
app.get('/users', logRequest, (req, res) => {  
  res.send('List of users');  
});
```

ExpressJS Routing

5. Express Router:

An isolated instance of middleware and routes, used to structure and organize the application.

Example:

Use `express.Router()` to create modular routers.

```
const express = require('express');  
const router = express.Router();
```

```
router.get('/', (req, res) => {  
  res.send('Router homepage');  
});
```

```
router.get('/about', (req, res) => {  
  res.send('About page');  
});
```

```
module.exports = router;
```


ExpressJS Routing

6. Express.js Route Redirects:

Redirect clients to a different route.

Example:

Redirect from one route to another.

```
app.get('/old-route', (req, res) => {  
  res.redirect('/new-route');  
});
```



Promise in JS

Promises are a powerful feature in JavaScript used to handle asynchronous operations. They provide a clean and structured way to work with asynchronous code, making it easier to manage.

A Promise in JavaScript represents the eventual completion or failure of an asynchronous operation and its resulting value. Promises have three states: pending, fulfilled, and rejected. They are especially useful when dealing with tasks like fetching data from an API, reading files, or making database queries.



Promise in JS example

```
// Function that returns a Promise to simulate fetching data
const fetchData = () => {
  return new Promise((resolve, reject) => {
    // Simulating an asynchronous operation (e.g., fetching data from
    an API)
    setTimeout(() => {
      // Simulating a condition to demonstrate resolving or rejecting
      the promise
      const shouldReject = Math.random() < 0.2; // 20% chance of
      rejection
```

Promise in JS example

```
if (shouldReject) {  
    reject(new Error('Failed to fetch data'));  
} else {  
    const data = { message: 'Data fetched successfully' };  
    resolve(data);  
}  
, 1000); // Simulating a delay of 1 second  
});  
};  
fetchData()  
    .then((data) => {  
        console.log(data.message); // Output: Data fetched successfully  
    })  
    .catch((error) => {  
        console.error(error.message); // Output: Failed to fetch data  
    });
```

async and await

async and await are features in JavaScript that simplify working with asynchronous code, making it look more like synchronous code.

An async function always returns a Promise. Inside an async function, you can use the await keyword to wait for a Promise to resolve.

Inside an async function, you can use await to pause the execution of the function until the Promise is resolved.

async and await

```
async function fetchDataWithAwait() {  
  try {  
    const data = await fetchData();  
    console.log(data.message); // Output: Data fetched successfully  
  } catch (error) {  
    console.error(error.message);  
  }  
}  
  
fetchDataWithAwait();
```

Using try and catch blocks, you can handle errors in a cleaner way compared to using .then() and .catch()

```
async function fetchDataWithTryCatch() {  
  try {  
    const data = await fetchData();  
    console.log(data.message);  
  } catch (error) {  
    console.error('Error:', error.message);  
  }  
}
```

```
fetchDataWithTryCatch();
```



Authentication and security with Passport.js

Passport.js is a lightweight, flexible authentication middleware for Node.js applications

Passport.js plays a crucial role in securing Node.js applications by managing the authentication process. It helps developers implement user authentication, ensuring that only authorized users can access protected resources.

Passport achieves this by providing a standardized way to handle authentication strategies, whether it's through local username/password, social logins (OAuth), or other methods.

Key features of Passport.js

- Extensibility
- Ease of Use
- Middleware Architecture
- Modularity

Authentication and security with Passport.js

Installation of Passport.js

To install the passport and authentication strategies use npm

Below is the command to install

```
npm install passport passport-local passport-google-oauth20
```

Authentication and security with Passport.js

Middleware Configuration

1. Set Up Passport Middleware
2. Initialize and Use Passport:

Call `passport.initialize()` and `passport.session()` to set up Passport for request processing.

Authentication and security with Passport.js

Example code

```
const express = require('express');  
const passport = require('passport');  
const app = express();  
// Middleware Setup  
app.use(passport.initialize());  
app.use(passport.session());
```



Authentication and security with Passport.js

Passport.js offers various authentication strategies to authenticate users in a Node.js application. Each strategy is designed to handle different authentication mechanisms, such as local authentication with usernames and passwords, OAuth authentication with third-party providers like Google or Facebook, and more.

1. Local Strategy:

The Local Strategy is the most basic authentication strategy provided by Passport.js.

It authenticates users using a username and password stored locally within the application's database.



Authentication and security with Passport.js

2. OAuth Strategies:

OAuth is an open standard for access delegation commonly used for authentication with third-party services.

Passport.js provides OAuth strategies for popular third-party authentication providers such as Google, Facebook, Twitter, GitHub, and more.



Authentication and security with Passport.js

3. JWT Strategy:

JSON Web Token (JWT) is a compact and self-contained way to transmit information securely between parties.

Passport.js provides a JWT Strategy for authenticating users based on JWTs.

In this strategy, JWTs containing user information are typically included in HTTP headers or request bodies, allowing applications to verify and authenticate users without the need for sessions or cookies.



Authentication and security with Passport.js

```
// Local strategy for username/password authentication
passport.use(new LocalStrategy({
  usernameField: 'email',
  passwordField: 'password'
}),
async (email, password, done) => {
  try {
    const user = await User.findOne({ email });
    if (!user) {
      return done(null, false, { message: 'Incorrect email.' });
    }
    if (!user.validPassword(password)) {
      return done(null, false, { message: 'Incorrect password.' });
    }
    return done(null, user);
  } catch (err) {
    return done(err);
  }
});
```




Error Handling and Logging

Error handling and logging are critical aspects of Node.js application development to ensure robustness, reliability, and maintainability.

1. Try-Catch Blocks:

Try-catch blocks provide a structured way to handle synchronous errors within JavaScript code. They allow you to gracefully manage exceptions that may occur during the execution of specific code blocks.

Error Handling and Logging

EXAMPLE

```
try {  
    // Code block where errors may occur  
    // Place code that may throw exceptions inside the try block  
} catch (error) {  
    // Code block to handle errors  
    // Catch block is executed if any exception is thrown within the try block  
    // 'error' is the exception object that contains information about the error  
}
```

Error Handling and Logging

2. Promises:

Promises provide a cleaner and more flexible approach to asynchronous error handling compared to error-first callbacks. They allow chaining multiple asynchronous operations and handling errors centrally.

```
somePromiseFunction()  
  .then(result => {  
    // Process the result  
  })  
  .catch(error => {  
    // Handle the error  
  });
```

Error Handling and Logging

3. Async/Await:

Async/await is a modern JavaScript feature that simplifies asynchronous error handling by allowing developers to write asynchronous code in a synchronous-like manner.

```
async function someAsyncFunction() {  
  try {  
    const result = await somePromiseFunction();  
    // Process the result  
  } catch (error) {  
    // Handle the error  
  }  
}
```



Error Handling and Logging

Logging:

Logging in Node.js is a fundamental practice for monitoring, debugging, and analyzing application behavior.

It involves capturing and recording relevant information, such as application events, errors, and user interactions, to facilitate troubleshooting and performance optimization.

Node.js offers several logging mechanisms and libraries to implement logging effectively.

Error Handling and Logging

Console Logging: Utilize the built-in console module for basic logging during development. Use methods like `console.log()`, `console.error()`, `console.warn()`, etc., to log information, errors, warnings, and debug messages.

Third-Party Logging Libraries: Consider using third-party logging libraries such as Winston, Bunyan, or Pino for more advanced logging capabilities. These libraries offer features like logging levels, custom formatting, log rotation, and integration with external logging services.



Error Handling and Logging

Logging Middleware: Implement logging middleware to log request and response information, including HTTP status codes, request methods, URLs, request bodies, and response bodies. This helps in debugging and monitoring application behavior.

Error Logging: Log errors and exceptions with relevant context information such as stack traces, error messages, timestamps, and request identifiers. This aids in diagnosing and troubleshooting issues in production environments.

× ○ DIGITAL LEARNING CONTENT



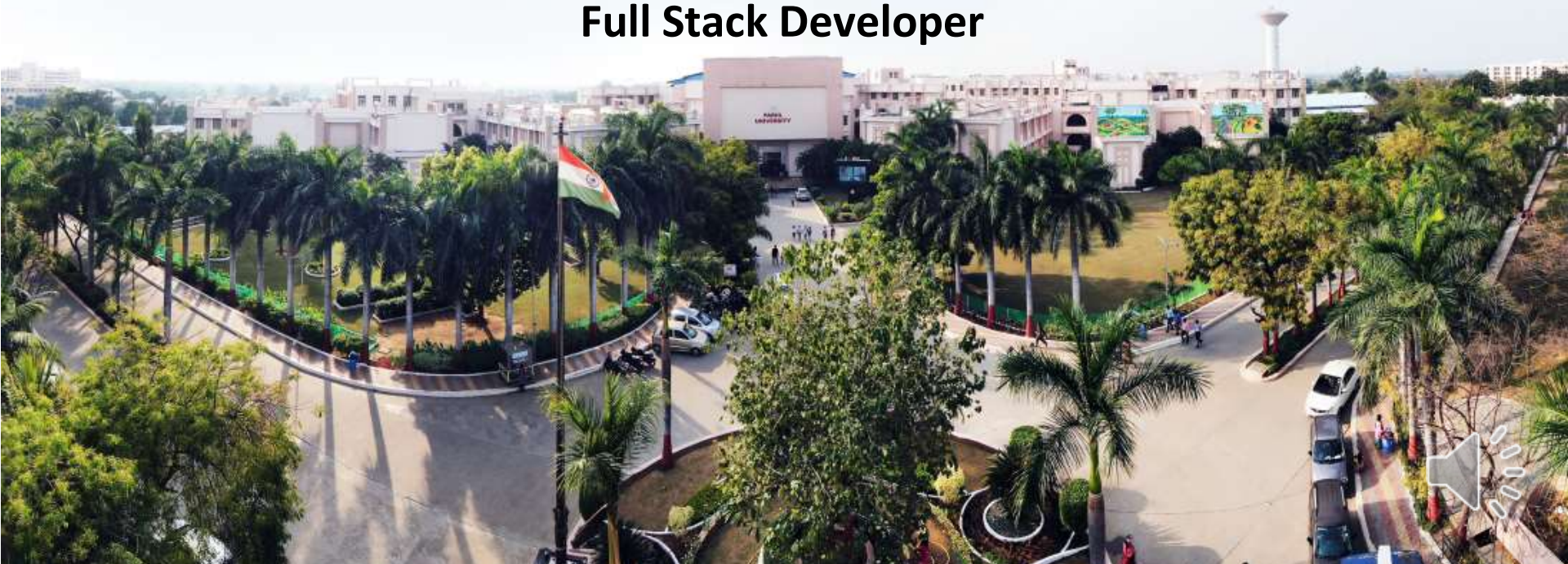
Parul[®] University



www.paruluniversity.ac.in

MEAN Full Stack

Amir Hussain
Cyber security trainer
&
Full Stack Developer





CHAPTER-4th

Introduction



What is Angular?

Angular is a development platform and framework for building single-page client applications using HTML and TypeScript. Angular is built using TypeScript.

Angular has many improvements and innovations over its predecessor, AngularJS. It is easy to learn and develop enterprise-scale applications. You can build extendable, maintainable, testable, and standardized applications using Angular.

Angular provides all the core functionality that you need to develop a client application. It comes bundled with a router module, an HTTP module, a forms API, etc. We can use it to build amazing client-side applications



Why to Learn Angular?

Community support: It is developed and maintained by Google. Also, it is based on typescript which is created by Microsoft. Hence it has a big community support.

Free of Cost: Since it is an open-source language, therefore developers are allowed to use its components and all methods for free.

Easy to learn: It has a big community and many resources are available
TypeScript: Angular is a typescript-based framework which is a statically typed superset of javascript.

Angular CLI: Angular CLI helps in simplification processes like creating, building, and deploying the application.

Cross-Platform Development: With NativeScript it is used to build dynamic mobile-based applications.



Applications of Angular

Single-Page Applications (SPAs): Angular is particularly well-suited for building SPAs, where a single webpage is created and content is dynamically updated as the user interacts.

Real-Time Applications: Angular is most used in real-time applications, such as chat applications, weather and forecasting, and live tracking systems, where immediate updates are essential.

Enterprise Applications: Angular is commonly used for developing large-scale enterprise applications. Its modular architecture, dependency injection, and TypeScript support make it suitable for building complex and maintainable applications.



contn...

- **Content Management Systems (CMS):** Angular can be used to build custom content management systems, providing a dynamic and responsive user interface for managing content.
- **E-commerce Websites:** Angular's two-way data binding and dynamic features make it suitable for building interactive e-commerce websites with real-time updates and user interactions.
- **Dashboards and Analytics Tools:** Angular is used to create interactive dashboards and analytics tools, it helps in data visualization and crucial real-time updates.



Key Features of Angular

- **Component-Based Architecture:** Angular applications are built using components, which are self-contained units of functionality that can be reused and composed to build complex user interfaces.
- **Two-Way Data Binding:** Angular uses two-way data binding to automatically synchronize the data between the model (business logic) and the view (UI), ensuring that changes in one are reflected in the other.
- **Dependency Injection:** Angular's dependency injection system allows components to be loosely coupled and promotes modular and testable code by providing dependencies to components as needed.



contn...

- **Routing:** Angular's built-in router allows developers to create single-page applications with multiple views and routes, enabling navigation between different parts of the application without page reloads.
- **Forms and Validation:** Angular provides powerful features for building forms and performing client-side validation, including built-in form controls, form validation directives, and custom validators.
- **HTTP Client:** Angular's built-in HTTP client module simplifies making HTTP requests to servers and handling responses, enabling seamless communication between the client and server.
- **Directives and Pipes:** Angular provides a variety of built-in directives and pipes for manipulating the DOM, transforming data, and creating reusable UI components



contn...

Before you start exploring the Angular platform, you should know about the Angular CLI. The Angular CLI is the fastest, easiest, and recommended way to develop Angular applications. The Angular CLI makes a number of tasks easy. Here are some examples:

- | | |
|--------------------|---|
| ng build | Compiles an Angular app into an output directory. |
| ng serve | Builds and serves your application, rebuilding on file changes. |
| ng generate | Generates or modifies files based on a schematic. |
| ng test | Runs unit tests on a given project. |



Build an App with Angular CLI

The Angular CLI (Command Line Interface) is a tool that helps developers quickly scaffold and build Angular applications. With the Angular CLI, developers can easily create new projects, generate components and services, and run development servers. we will see the process of building an app with Angular and the Angular CLI by creating a simple application that showcases an example.

Before we proceed to build the application, we need to install the Angular CLI on the local system.



Steps for creating the building App using Angular CLI

Step 1: First, we will need to have Node.js and the Node Package Manager(NPM) installed on our computer. Download the Node.js from the official website.

Step 2: After successful installation of Node.js & NPM, install the Angular CLI by running the below command in your terminal.

```
npm install -g @angular/cli
```



contn...

Step 3: Once we have the Angular CLI installed, we can use the CLI to create a new Angular project. Now, in order to check whether the Angular CLI is properly installed or not, we can check for their version installed by typing the below command:

```
node -v  
npm -v
```

Step 4: Open the terminal and navigate to the directory where you want to create your project. Run the below command where “my-first-app” is the name of your project. This will create a new Angular project in a new directory called “my-first-app”.

```
ng new my-first-app
```

contrn...

Step 5: Next, navigate into the project directory by running the following command:

```
cd my-first-app
```

Step 6: Open the application in the browser using the following command:

```
ng serve
```

Get familiar with your Angular application

The application source files that is `src/app`. Key files that the CLI generates automatically include the following:

`app.module.ts`: Specifies the files that the application uses. This file acts as a central hub for the other files in your application.

`app.component.ts`: Also known as the class, contains the logic for the application's main page.

`app.component.html`: Contains the HTML for AppComponent. The contents of this file are also known as the template. The template determines the view or what you see in the browser.

contrn...

app.component.css: Contains the styles for AppComponent. You use this file when you want to define styles that only apply to a specific component, as opposed to your application overall.

A component in Angular is made up of three main parts—the template, styles, and the class. For example, **app.component.ts**, **app.component.html**, and **app.component.css** together constitute the AppComponent.

This structure separates the logic, view, and styles so that the application is more maintainable and scalable.

contr...

Explore the different files and folders within the project structure, including:

src/app: Contains the application components, modules, services, and other files.

src/assets: Contains static assets such as images, fonts, and stylesheets.

src/index.html: The main HTML file for the application.

angular.json: Angular project configuration file.

package.json: npm package configuration file.

example

```
import { Component } from "@angular/core";

@Component({
  selector: "app-item",
  // the following metadata specifies the location of the other parts of
  the component
  templateUrl: "./item.component.html",
  styleUrls: ["./item.component.css"],
})
export class ItemComponent {
  // your code goes here
}
```

Angular Components

The Angular Component is the main building block of an Angular application.

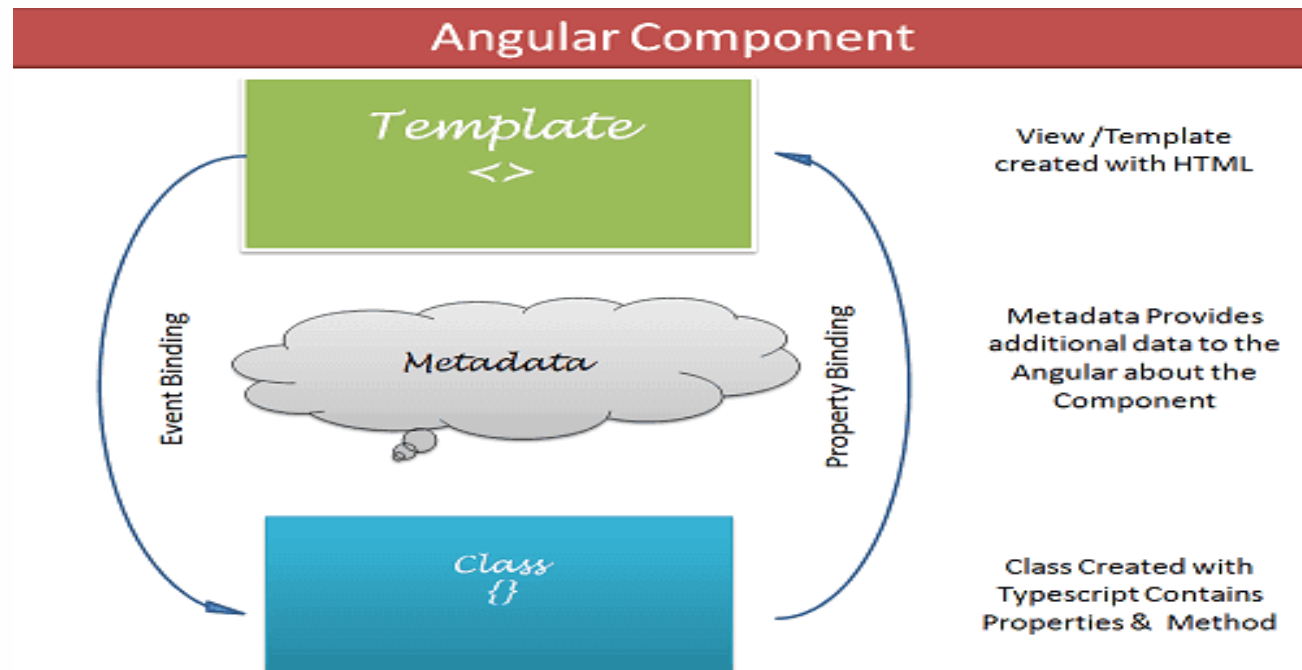
The Component is responsible for providing the data to the View. Angular uses data binding to get the data from the Component to the View and vice versa.

Angular applications can have lots of components. Each Component handles a small part of the UI. These components work together to produce the complete user interface of the application.

Angular Components

The Components consist of three main building blocks

Template
Class
MetaData



Anatomy of Angular Components

1. Template (View)

The Template defines the layout and content of the View. Without the template, there is nothing for Angular to render to the DOM.

The Templates are only HTML codes and the Angular-specific special HTML markups (known as the Angular Template Syntax)

You can add Angular directives, Angular Pipes & Other Angular Components on the template.

Angular Components

There are two ways you can specify the Template in Angular.

- 1. Defining the Template Inline**
- 2. Provide an external Template**

Anatomy Angular Components

2.Component Class:

The component class is a TypeScript class that contains the logic and behavior of the component.

It typically defines properties, methods, and lifecycle hooks for managing the component's state and behavior.

3. CSS (Style)

Contain css for styling

Anatomy Angular Components

Component Metadata:

The component metadata is defined using the `@Component` decorator, which provides configuration options for the component. It includes properties such as `selector`, `templateUrl` (or `template`), `styleUrls` (or `styles`), and more.

Angular Components

To create a new component in Angular, you can use the Angular CLI (Command Line Interface) or manually create the necessary files and code.

ng generate component <component-name>

Using in Templates:

In an HTML template file, you can use the component's selector as a custom HTML element to render the component:

```
<app-my-component></app-my-component>
```


lifecycle hooks

Angular provides several lifecycle hooks that allow you to tap into different stages of a component's lifecycle, from initialization to destruction. These hooks enable you to perform custom logic at specific points in the component's lifecycle

1. **ngOnInit:**

Called once when the component is initialized.
Useful for initializing component properties, fetching initial data, or performing setup tasks

lifecycle hooks

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-example',
  template: `{{ message }}`,
})
export class ExampleComponent implements OnInit {
  message: string;

  ngOnInit() {
    this.message = 'Hello, Angular!';
  }
}
```

lifecycle hooks

ngOnChanges:

Called every time an input property of the component changes.

Useful for responding to changes in input properties and performing calculations or updates based on those changes.

ngOnDestroy:

Called once before the component is destroyed.

Useful for performing cleanup tasks, such as unsubscribing from observables or releasing resources.

lifecycle hooks

These are the hooks for components or directives, in call order:

constructor()
OnChanges
OnInit
DoCheck
OnDestroy

And these are the hooks for a component's children components:

AfterContentInit
AfterContentChecked
AfterViewInit
AfterViewChecked

lifecycle hooks

```
export class LifecycleExampleComponent implements OnChanges, OnInit,
OnDestroy {
  @Input() inputValue: string;
  message: string = "";

  ngOnChanges(changes: SimpleChanges) {
    console.log('ngOnChanges called:', changes);
    if (changes.inputValue) {
      this.message = 'Input value changed to: ' + changes.inputValue.currentValue;
    }
  }
  ngOnInit() {
    console.log('ngOnInit called');
    // Initialization logic
  }
  ngOnDestroy() {
    console.log('ngOnDestroy called');
    // Cleanup logic
  }
}
```

Angular Components

Example

```
// app.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'Angular Components Example';
}
```

Child Components

In Angular, a child component is a component that is nested within another component, known as its parent component. Child components are a fundamental concept in Angular's component-based architecture and are used to create modular and reusable UI elements within an application.

Modules

Modules in Angular are defined using the NgModule decorator, and they are TypeScript classes annotated with @NgModule.

Angular applications typically have at least one root module, conventionally named AppModule, which serves as the entry point for the application.

Modules

```
import { NgModule } from '@angular/core';  
import { BrowserModule } from '@angular/platform-browser';  
import { AppComponent } from './app.component';
```

```
@NgModule({  
  declarations: [  
    AppComponent,  
  ],  
  imports: [  
    BrowserModule,  
  ],  
  providers: [],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

Modules

How Modules Work:

Declaration: Modules declare which components, directives, and pipes belong to them through the declarations array.

Import: Modules can import other modules or Angular libraries via the imports array.

Providers: Modules can provide services that will be available throughout the application through the providers array.

Bootstrap: The bootstrap array specifies the root component that Angular should bootstrap when it starts the application.

Modules

Pros of Using Modules:

Modularity: Modules promote modularity by organizing related components, directives, services, and pipes into cohesive units.

Reusability: Modules can be reused across different parts of the application or even in multiple applications.

Encapsulation: Modules encapsulate functionality, making it easier to manage and maintain the application's codebase.

Lazy Loading: Angular's modular structure allows for lazy loading of modules, improving performance by loading modules asynchronously when needed.



Services

In Angular, a service is a class that encapsulates a specific functionality and can be shared across multiple components or other services within an Angular application.

Services are typically used for tasks such as data fetching from APIs, business logic, state management, and more. They promote code reusability, modularity, and maintainability in Angular applications.

Services

How Services Work

Services in Angular are singleton objects, meaning there is only one instance of each service created and shared throughout the application.

When a service is injected into a component or another service via Angular's dependency injection system, Angular provides the same instance of the service everywhere it is injected. This ensures consistency and avoids unnecessary duplication of resources.

.

Why we use Services

Code Organization: Services help in organizing and separating concerns in Angular applications, keeping components lean and focused on presentation logic.

Reusability: Services promote code reusability by encapsulating common functionality that can be shared across multiple components.

Maintainability: Services make it easier to maintain and test Angular applications by encapsulating business logic, data fetching, and other functionalities in a modular way.

Dependency Injection: Angular's dependency injection system makes it easy to inject services into components, promoting loose coupling and making the code more testable and maintainable.



Services Example

```
import { Injectable } from '@angular/core';
```

```
@Injectable()
```

```
export class StudentService {
```

```
  getStudents(): any[] {
```

```
    return [
```

```
      {
```

```
        ID: 'std101', FirstName: 'Preety', LastName: 'Tiwary',
```

```
        Branch: 'CSE', DOB: '29/02/1988', Gender: 'Female'
```

```
      },
```

```
      {
```

```
        ID: 'std105', FirstName: 'Sambit', LastName: 'Satapathy',
```

```
        Branch: 'CSE', DOB: '12/94/1991', Gender: 'Male'
```

```
      }
```

```
    ];
```

```
  }
```

```
}
```

Using the Service in Angular

```
-- Other Import statements  
import {StudentService} from './student.service';
```

Import the service

```
@Component({  
  -- Other Properties  
  
  providers:[StudentService]  
  
})
```

Register the service

```
export class AppComponent {  
  students: any[];  
  
  constructor(private _studentService: StudentService) {  
    this.students = this._studentService.getStudents();  
  }  
  
}
```

Use the service

Services

Angular provides several built-in services that offer functionality commonly needed in Angular applications. Some of the commonly used built-in services include:

HttpClient: Used for making HTTP requests to servers.

Router: Provides the functionality for routing/navigation within an Angular application.

ActivatedRoute: Represents the active route in the application's route hierarchy.

FormBuilder: Provides methods for dynamically creating and managing Angular forms.

InjectionToken: Used for creating custom injection tokens for dependency injection.

Services

Custom services are services created by developers to encapsulate specific functionality required by their Angular applications.

These services are typically created using the `@Injectable()` decorator and can be provided at different levels of the application (e.g., root, module, component).

Custom services can encapsulate business logic, data fetching from APIs, state management, utility functions, and more.

Services

- 1. Singleton Services (provided in root):**
- 2. Module-level Services (provided in feature or shared modules):**
- 3. Component-level Services (provided in component metadata):**
- 4. Custom Injectors (Hierarchical Injector):**

Data Binding

Data binding in Angular is a powerful feature that allows you to establish a connection between the component class (the model) and the template (the view), ensuring that data changes in one place are automatically reflected in the other.

Angular supports various types of data binding, including,

- interpolation**
- property binding**
- event binding**
- and two-way binding**



Data Binding

1 Interpolation

Interpolation is the simplest form of data binding in Angular. It allows you to embed expressions in double curly braces ({{ }}) within your HTML template, and Angular replaces those placeholders with the result of the expression.

syntax: {{variable}}

example <p>{{name}}</p>

** name is a variable in .ts file



Data Binding

2. Property Binding

Property binding allows you to set an element property to the value of a component's property. It uses square brackets [] to bind a property of an HTML element to a value in the component.

Template

```
<img [src]="imageUrl" alt="Image">  
<button [disabled]="isDisabled">Click me</button>
```

Component Class

```
imageUrl = 'https://example.com/image.jpg';  
isDisabled = true;
```

Data Binding

3. Event Binding

Event binding allows you to listen for events (e.g., click, change) emitted by HTML elements and execute a method in the component when the event occurs. It uses parentheses () to bind an event to a method in the component class

Template

```
<button (click)="onButtonClick()">Click me</button>
```

component class

```
onButtonClick() {  
  console.log('Button clicked');  
}
```



Data Binding

4. Two way Data Binding

Two-way data binding in Angular is a powerful feature that allows you to bind data in both directions between the component class (model) and the HTML template (view).

This means that changes in the model update the view, and changes in the view update the model automatically, ensuring that the data remains synchronized between the two.

1. Data Flow:

With two-way data binding, data flows in both directions:

From the component class to the template (model → view)

From the template to the component class (view → model)

Data Binding

2. Event & Property Binding:

Two-way data binding combines event binding and property binding to achieve bi-directional data flow.

It uses the [(ngModel)] directive, which is a combination of square brackets ([]) for property binding and parentheses (()) for event binding.

syntax:

```
<input [(ngModel)]="property">
```

Data Binding

Example of Two way data Binding.

template:

```
<input [(ngModel)]="username" placeholder="Enter your name">  
<p>Hello, {{ username }}!</p>
```

Component Class

```
export class AppComponent {  
  username: string = ""; // Component property bound to input field  
}
```

Angular template

In Angular, a template is a combination of HTML markup and Angular-specific syntax that defines the user interface (UI) of a component.

Templates allow you to define the structure and layout of your application's UI by incorporating dynamic content, data binding, directives, and event handling.

Templates play a crucial role in Angular applications as they determine how the component's data is presented to the user.

Angular template

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-root',  
  template: `  
    <h1>Welcome to My Angular App!</h1>  
    <ul>  
      <li *ngFor="let item of items">{{ item }}</li>  
    </ul>  
  `,  
})  
export class AppComponent {  
  items: string[] = ['Item 1', 'Item 2', 'Item 3'];  
}
```

Angular Directives

Angular provides several built-in structural directives, including `ngIf`, `ngFor`, and `ngSwitch`, which are commonly used in templates to conditionally render or manipulate DOM elements.

1. `ngIf`

The `ngIf` directive conditionally includes or excludes an element from the DOM based on the truthiness of an expression.

```
<div *ngIf="isLoggedIn">  
  <p>Welcome, {{ username }}!</p>  
</div>
```

Angular Directives

2. ngFor

The ngFor directive is used to loop over a collection of items and render a template for each item in the collection.

```
<ul>  
  <li *ngFor="let item of items">{{ item }}</li>  
</ul>
```

3. ngSwitch

```
<div [ngSwitch]="condition">  
  <p *ngSwitchCase="case1">Case 1 Content</p>  
  <p *ngSwitchCase="case2">Case 2 Content</p>  
  <p *ngSwitchDefault>Default Content</p>  
</div>
```

Angular Forms and Validation

Forms are used to handle user input data. Angular supports two types of forms.

They are Template driven forms and Reactive forms.

1. Template driven forms

Template driven forms is created using directives in the template. It is mainly used for creating a simple form application.

Angular Forms and Validation

Template-driven forms are best if you want to add a simple form to your application. For example: email list signup form.

Template-driven forms are easy to use in the application but they are not as scalable as Reactive forms.

Template-driven forms are mainly used if your application's requires a very basic form and logic. It can easily be managed in a template.

Angular Forms and Validation

2. Reactive forms

Reactive Forms is created inside component class so it is also referred as model driven forms.

Every form control will have an object in the component and this provides greater control and flexibility in the form programming.

Reactive forms are defined programmatically in the component class using TypeScript.

Angular Forms and Validation

```
export class ReactiveFormComponent {  
  form: FormGroup;  
  
  constructor(private fb: FormBuilder) {  
    this.form = this.fb.group({  
      name: ['', [Validators.required, Validators.minLength(3)]],  
      email: ['', [Validators.required, Validators.email]]  
    });  
  }  
}
```

Angular Forms and Validation

```
<form [formGroup]="form" (ngSubmit)="onSubmit()">
  <div>
    <label for="name">Name:</label>
    <input type="text" id="name" formControlName="name">
  </div>
  <div>
    <label for="email">Email:</label>
    <input type="email" id="email" formControlName="email">
  </div>

  <button type="submit" [disabled]="form.invalid">Submit</button>
</form>
```

Need of form validation

Angular form validation is crucial for ensuring that the data submitted by users through forms meets specific requirements and constraints.

Validating user input is essential for maintaining data integrity, improving user experience, and preventing security vulnerabilities.

There are several types of validation techniques that can be employed to ensure that user input meets specific criteria and constraints.

These validation techniques can be applied in both template-driven forms and reactive forms.

Form validation

1. Built-in Validators:

Angular provides a set of built-in validators that can be used to perform common validation tasks.

These validators are available as static methods in the Validators class and can be applied to form controls in template-driven and reactive forms.

required: Ensures that a form control has a non-empty value.

minLength and **maxLength:** Validates the minimum and maximum length of the input value.

Form validation

pattern: Validates the input value against a regular expression pattern.

email: Validates the input value as an email address.

min and max: Validates numeric input values against a minimum and maximum value.



Form validation

```
<form #registrationForm="ngForm" (ngSubmit)="onSubmit(registrationForm)">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" ngModel required minlength="3">
  <div *ngIf="registrationForm.controls.name.invalid &&
(registrationForm.controls.name.dirty ||registrationForm.controls.name.touched)">
    <div *ngIf="registrationForm.controls.name.errors?.required">Name is
required.</div>
    <div *ngIf="registrationForm.controls.name.errors?.minlength">Name must be
at least 3 characters long.</div>
  </div>
```



Form validation

```
<label for="email">Email:</label>
  <input type="email" id="email" name="email" ngModel required email>
  <div *ngIf="registrationForm.controls.email.invalid &&
(registrationForm.controls.email.dirty || registrationForm.controls.email.touched)">
    <div *ngIf="registrationForm.controls.email.errors?.required">Email is
required.</div>
    <div *ngIf="registrationForm.controls.email.errors?.email">Invalid email
format.</div>
  </div>
```


Form validation

2. Custom Validators:

Custom validators allow developers to define their own validation logic tailored to specific requirements. These validators are implemented as functions that accept a form control as an argument and return either null if the validation passes or a validation error object if it fails.

Form validation

```
import { AbstractControl, ValidatorFn, ValidationErrors } from
 '@angular/forms';

export function passwordValidator(control: AbstractControl):
 ValidationErrors | null {
  const hasUpperCase = /[A-Z]/.test(control.value);
  const hasLowerCase = /[a-z]/.test(control.value);
  const hasDigit = /[0-9]/.test(control.value);

  if (!hasUpperCase || !hasLowerCase || !hasDigit) {
    return { invalidPassword: true };
  }

  return null;
}
```

Example of reactive form

```
<form [formGroup]="myForm" (ngSubmit)="onSubmit(myForm)">
  <div>
    <label>
      Name:
      <input formControlName="name" placeholder="Your name">
    </label>
  </div>
  <div>
    <label>
      Email:
      <input formControlName="email" placeholder="Your email">
    </label>
  </div>
</form>
```

Example of reactive form

```
export class AppComponent implements OnInit {  
  myForm: FormGroup;  
  
  constructor(private fb: FormBuilder) {}  
  
  ngOnInit() {  
    this.myForm = this.fb.group({  
      name: ['Sammy', Validators.required],  
      email: ['', [Validators.required, Validators.email]],  
      message: ['', [Validators.required, Validators.minLength(15)]],  
    });  
  }  
}
```

Routing and Navigation

Angular Routing and navigation enable the creation of single-page applications (SPAs) where users can navigate between different views or pages without the need for full page reloads.

Even though a single page application (SPA) does not have multiple page concept, it does moves from one view (list of expenses) to another view (expense details).

The process of defining navigation element and the corresponding view is called Routing.

Angular provides a separate module, RouterModule to set up the navigation in the Angular application.

Routing and Navigation

<router-outlet> is a router directive from the router library which is used for directing the router where to display the routed views. To use this directive, you need to first import it in app.module.ts file from the angular router directory using the following command:

```
import { Routes, RouterModule } from '@angular/router';
```

RouterLink:

a single page application (SPA) does not have different pages to link to. Instead, it has different views to display to the user. To allow a user to navigate and change the view, you will want to use the RouterLink directive instead of href.

Routing and Navigation

`<a routerLink="/users/sammy"> Link that uses a string.</a>`

`<a [routerLink]="['/users', userId]">User Details</a>`

`<a routerLink="/home" routerLinkActive="active">Home</a>`

```
navigateToDashboard() {  
  this.router.navigate(['/dashboard']);  
}
```

Configure Routing in angular

Step1: The Angular Router enables navigation from one view to the next as users perform application tasks.

First, we need to import the routing modules inside our app.module.ts file.

```
import { RouterModule } from '@angular/router';
```

```
imports: [  
  BrowserModule, RouterModule  
],
```


Configure Routing in angular

Step2: Make routes in Route.ts file

```
const appRoutes: Routes = [  
  { path: '/',  
    component: HomeComponent  
  },  
  {  
    path: 'about',  
    component: AboutComponent  
  },  
  { path: 'contacts/:id', component: ContactDetailComponent },  
  { path: '**', redirectTo: '/' }  
];  
export default appRoutes;
```

Configure Routing in angular

Step3: Import router module in module.ts file

```
@NgModule({  
  declarations: [  
    AppComponent,  
    HomeComponent,  
    AboutComponent,  
    DashboardComponent  
  ],  
  imports: [  
    BrowserModule, RouterModule  
  ],  
  providers: [],  
  bootstrap: [AppComponent]  
})
```



Configure Routing in angular

Step3: Add routerLinks in component.html file

```
<div style="text-align:center">  
  <h1>  
    Welcome to {{title}}!!  
  </h1>  
  <nav>  
    <a routerLink="home" routerLinkActive="active">Home</a>  
    <a routerLink="about">About</a>  
    <a routerLink="dashboard">Dashboard</a>  
  </nav>  
  <router-outlet></router-outlet>  
</div>
```



Route Guard

A route guard is a feature of the Angular Router that allows developers to run some logic when a route is requested, and based on that logic, it allows or denies the user access to the route. It's commonly used to check if a user is logged in and has the authorization before he can access a page.

```
class MyGuard implements CanActivate {  
  canActivate() {  
    return true;  
  }  
}
```

Defining route in Route.ts file

```
{  path:      'contacts/:id',    canActivate:[MyGuard],    component:  
ContactDetailComponent}
```

HTTP and observables

HTTP (Hypertext Transfer Protocol) is the foundation of data communication on the World Wide Web. In Angular, HTTP is used to request and send data from a server

HTTP observables in Angular play a crucial role in handling asynchronous communication with backend servers.

The Angular HttpClient module provides methods for making HTTP requests, such as GET, POST, PUT, DELETE, etc., and it returns observables for handling responses asynchronously.

Observables

Observables simplify working with asynchronous data in Angular.

They provide a way to listen to data and events over time, making them a natural choice for HTTP requests, which are inherently asynchronous.

1. Importing HTTP client Module

```
import { HttpClientModule } from '@angular/common/http';
```

```
@NgModule({  
  imports: [HttpClientModule],  
})
```

Observables

2. configuring HttpClientModule

```
import { HttpClient } from '@angular/common/http';
```

```
constructor(private http: HttpClient) {}
```

3. Making get request

```
getData() {  
  this.http.get('https://api.example.com/data')  
    .subscribe(data => {  
      console.log('Received data:', data);  
    });  
}
```

Observables

4. making Post request

```
sendData(data: any) {  
  this.http.post('https://api.example.com/data', data)  
    .subscribe(response => {  
      console.log('Server response:', response);  
    });  
}
```


× ○ DIGITAL LEARNING CONTENT



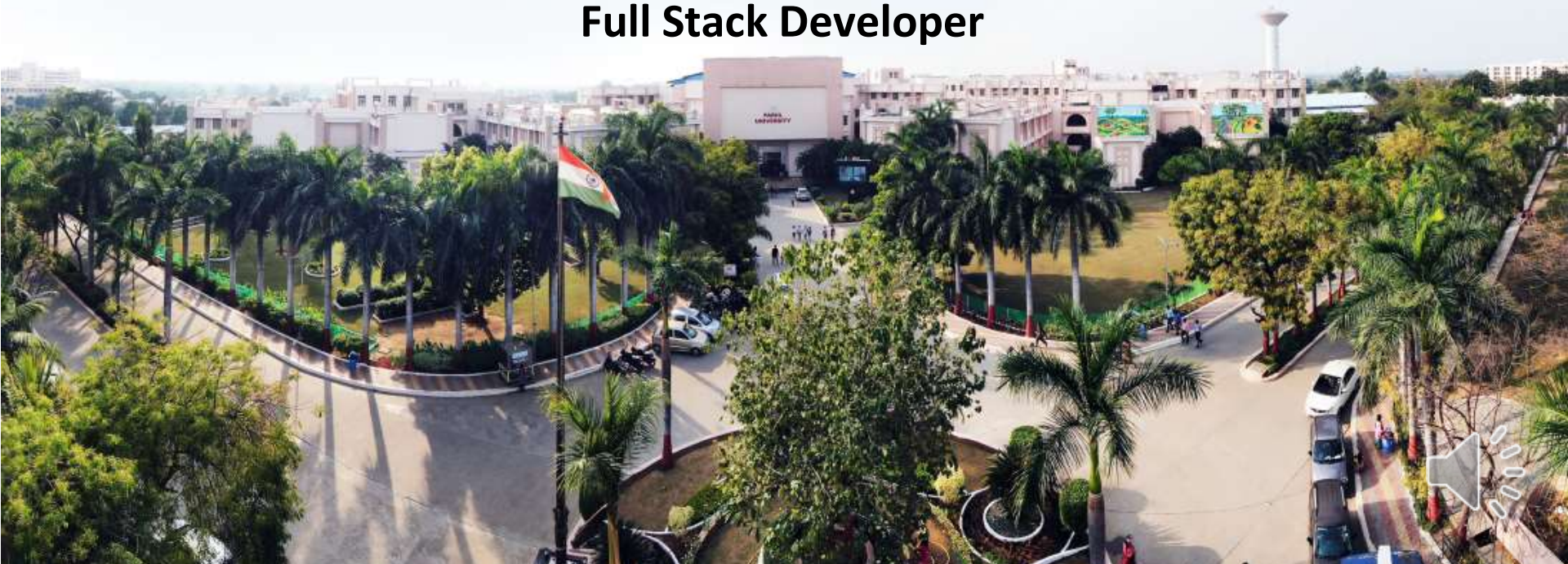
Parul[®] University



www.paruluniversity.ac.in

MEAN Full Stack

Deepak Upadhyay
Cyber security trainer
&
Full Stack Developer





CHAPTER-5th

Introduction



Integrating the Angular frontend with the Express.js APIs

- Integrating Angular frontend with an Express.js API is a common practice for building single-page applications (SPAs). Here's a breakdown of the key steps involved:

1. Setup:

- **Angular Frontend:** Use the Angular CLI to create a new project or work with an existing one. Make sure it's running development or production builds as needed.
- **Express.js API:** Create a new Express project using npm or yarn. Install any necessary dependencies for your API functionalities.



Integrating the Angular frontend with the Express.js APIs

2. Communication:

- HTTP Services:** Use Angular's HTTP Client for making API calls to your Express server. Define services to handle fetching, creating, updating, and deleting data from specific endpoints.
- CORS Configuration:** Ensure your Express server allows CORS (Cross-Origin Resource Sharing) requests from your Angular app's domain. This is often achieved with middleware like `cors`.

3. Data Mapping:

- Interfaces:** Define TypeScript interfaces for data models expected from your API responses. This ensures type safety and facilitates easier data handling.



Integrating the Angular frontend with the Express.js APIs

- Observables:** Use Observables within your services to handle asynchronous data responses from the API. Angular components can subscribe to these Observables to update their UI dynamically.

4. Deployment:

- Separate deployments:** You can deploy your Angular frontend and Express.js API separately, ensuring you configure DNS and server routing to serve the appropriate assets.
- Combined deployment:** Some tools like Nx Workspace allow combined builds and deployments for easier management, but might be more complex to set up initially.



Authentication and user management integration

- Authentication and User Management Integration for Angular & Express.js
- Integrating authentication and user management into your Angular frontend and Express.js API adds another layer of complexity. Here's a simplified breakdown:

Imagine:

- **Users:** People interacting with your website.
- **Security Guard:** Checking IDs (credentials) before letting users in.
- **Roles & Permissions:** Like VIP passes, giving users access to different parts of the storehouse (API).

Authentication and user management integration

Steps:

- 1.Login:** Users enter credentials (username/password) on the Angular frontend.
- 2.Security Guard:** Angular sends credentials to the Express API.
- 3.Verify:** The API checks credentials against a user database (internal or external).
- 4.Grant Access:** If valid, the API creates a "token" (like a VIP pass) and sends it back.
- 5.Secure Communication:** Angular stores the token securely and uses it with future API calls.
- 6.Access Control:** The API checks the token for validity and user roles/permissions on every request.
- 7.Restrict Access:** Only authorized users get access to specific data or actions.



Authentication and user management integration

Additional points:

- Token Storage:** Securely store the token on the frontend (e.g., browser storage with encryption).
- Token Expiration:** Set tokens to expire after a certain time for added security.
- Refresh Tokens:** Implement mechanisms to refresh expiring tokens without re-login.
- User Management:** Build features for user registration, profile management, etc. (in both frontend and API).



Handling real-time data with WebSockets

- Integrating WebSockets into your Angular frontend and Express.js API can unlock real-time data updates like chat, live dashboards, and notifications. Here's a simplified breakdown:

Imagine:

- WebSockets:** Like a direct, open line of communication between the website and the storehouse.
- Real-time Updates:** Like a live TV feed, showing things instantly as they change.

Steps:

- 1.Connect:** Angular opens a WebSocket connection to the Express API.
- 2.Listen:** Both sides stay connected, ready to send and receive messages.



Handling real-time data with WebSockets

3. Send Updates: The API pushes real-time data updates through the WebSocket to Angular.

4. Display Updates: Angular immediately updates the website with the new data.

Additional points:

- **Two-way communication:** Both Angular and the API can send messages for more interactive features.
- **Scalability:** Choose libraries and techniques that handle many concurrent connections efficiently.
- **Event-driven:** Define specific events (e.g., new message, data change) for the API to send updates.
- **Security:** Secure the WebSocket connection to prevent unauthorized access or data modification.



Error handling and testing

- Both **error handling** and **testing** are crucial practices in software development, working together to create reliable and user-friendly applications. Let's break them down:

Error Handling:

- **What it is:** The process of anticipating, identifying, and gracefully handling errors that occur during software execution.
- **Why it's important:** Prevents crashes, ensures program stability, provides meaningful feedback to users, and helps developers identify and fix problems efficiently.

Error handling and testing

•Key aspects:

- **Identifying errors:** Using mechanisms like try-catch blocks, assertions, and input validation to detect errors.
- **Handling errors:** Providing informative error messages to users, logging errors for debugging, and taking appropriate actions like retrying operations or providing alternative data.
- **Security:** Preventing sensitive information leaks or security vulnerabilities due to errors.



Error handling and testing

Testing:

- **What it is:** The process of verifying the correctness and functionality of software through various techniques.
- **Why it's important:** Ensures software meets requirements, catches bugs early, reduces risk of deploying defective code, and provides confidence in software quality.

Types of testing:

- **Unit testing:** Testing individual units of code (e.g., functions, classes) in isolation.
- **Integration testing:** Testing how different parts of the software interact with each other.
- **End-to-end testing:** Testing the entire system from the user's perspective, simulating real-world interactions.
- **Other types:** Performance testing, security testing, etc.

Error handling and testing

Relationship between Error Handling and Testing:

- **Error handling helps testing:** Errors caught during testing help improve error handling mechanisms.
- **Testing validates error handling:** Tests verify that error handling works as intended, providing feedback on its effectiveness.

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.in

MEAN Full Stack

Amir Hussain
Cyber security trainer
&
Full Stack Developer





CHAPTER-6th

Introduction



Preparing the application for deployment

1. Frontend (Angular):

- Build: Use `ng build --prod` to create an optimized production build.
- Minification: Minify and uglify your JavaScript and CSS files for smaller size and faster loading.
- Versioning: Update Angular and its dependencies to the latest stable versions for security and performance.
- Routing: Ensure your routing configuration works correctly in the deployment environment.



Preparing the application for deployment (count..)

2. Backend (Express.js):

- Dependencies: Install production dependencies only using `npm install --production`.
- Environment variables: Use environment variables to store sensitive information like database credentials, and configure them appropriately for the deployment environment.
- Error handling: Implement proper error handling and logging for production robustness.
- Scalability: Consider using a load balancer and process manager (e.g., PM2) for handling increased traffic.



Preparing the application for deployment (count...)

3. Database (MongoDB):

- Backups: Set up regular backups of your database to prevent data loss.
- Authentication: Implement authentication and authorization mechanisms to ensure data security.
- Indexing: Optimize database indexes for improved query performance.
- Replica sets: Consider using replica sets for high availability and disaster recovery.



Preparing the application for deployment (count...)

4. Deployment:

- Platform: Choose a suitable deployment platform like Heroku, AWS, or DigitalOcean, considering factors like scalability, cost, and ease of use.
- CI/CD: Implement a Continuous Integration/Continuous Delivery (CI/CD) pipeline for automated testing and deployment.
- Monitoring: Set up monitoring tools to track application performance and health.
- Security: Implement security measures like firewalls and intrusion detection systems.



Hosting and server setup options

- When it comes to choosing a hosting solution for your MEAN stack application, several options exist, each with its own advantages and disadvantages. Here's a breakdown of some popular options:
1. Managed Platforms:
 - Pros: Easy setup, automatic scaling, regular updates, security patches, managed databases, often offer support and monitoring.
 - Cons: Can be more expensive than self-managed options, limited customization, vendor lock-in.
 - Examples: Heroku, AWS Elastic Beanstalk, Azure App Service.



Hosting and server setup options

2. Cloud Providers:

- Pros: High scalability, flexible pricing, wide range of services and configurations, control over infrastructure.
- Cons: Requires more technical expertise to set up and manage, can be complex for beginners.
- Examples: Amazon Web Services (AWS), Google Cloud Platform (GCP), Microsoft Azure.



Hosting and server setup options

3. Virtual Private Servers (VPS):

- Pros: More control than shared hosting, affordable option for moderate traffic, root access for customization.
- Cons: Requires server management skills, security is your responsibility, scaling can be manual.
- Examples: DigitalOcean, Linode, Vultr.



Hosting and Server Setup Options (Count...)



Hosting and Server Setup Options (Count...)

4. Self-hosting:

- Pros: Full control and customization, cost-effective for high traffic applications.
- Cons: Requires significant technical expertise, ongoing maintenance and security updates, potential hardware costs.
- Examples: Setting up your own server on dedicated hardware or cloud infrastructure.



Security Best Practices

General:

- Stay up-to-date: Regularly update all components (Node.js, MongoDB, Express.js, Angular, libraries) to patch known vulnerabilities.
- Secure communication: Use HTTPS to encrypt communication between client and server.
- Input validation: Sanitize and validate all user input to prevent injection attacks (XSS, SQLi). Consider libraries like validator or express-validator.
- Authentication and authorization: Implement robust mechanisms for user authentication (e.g., JWT, OAuth) and authorization (e.g., role-based access control) to restrict unauthorized access.
- Error handling: Implement secure error handling to avoid exposing sensitive information in error messages.
- Vulnerability scanning: Regularly scan your application for vulnerabilities using tools like snyk or npm audit.



Security Best Practices

Node.js & Express.js:

- Use security libraries: Utilize libraries like helmet to configure security headers (e.g., Content Security Policy, X-XSS-Protection).
- Secure dependencies: Only use trusted dependencies from npm and avoid known vulnerable packages.
- Secure environment variables: Don't store sensitive information like database credentials in plain text. Use environment variables and keep them secure.



Security Best Practices

MongoDB:

- Enable authentication: Enforce user authentication and authorization for access to your MongoDB database.
- Use strong passwords: Implement strong password policies and store passwords securely (hashed and salted).
- Network access control: Limit access to your MongoDB database to only authorized IP addresses.
- Encryption: Consider encrypting data at rest and in transit for additional security.



Performance optimization

Performance optimization and testing are crucial aspects of deploying MEAN (MongoDB, Express.js, Angular, Node.js) applications to ensure they run smoothly and efficiently.

Optimize client-side performance by bundling and minifying assets, leveraging code splitting, and implementing lazy loading for resource-intensive components.

Employ server-side caching mechanisms to reduce database load and improve response times for frequently accessed data.



Performance optimization

Frontend Performance Optimization

- Code Splitting
- Lazy Loading Components
- Minification and Compression
- Client-Side Caching
- Optimizing Rendering Performance



Performance optimization

Backend Performance Optimization

- **Caching Strategies:** Implementing caching at various levels, such as in-memory caching, Redis caching, or Content Delivery Network (CDN) caching, helps reduce the number of database queries and API calls, resulting in faster response times.
- **Optimizing Database Queries:**
- **Reducing Server-Side Processing:**
- **Reduce middlewares**
- **Data Compression**

Performance optimization

Database Performance

- **Choosing the Right Indexes**
- **Compound Indexes**
- **Query Analysis with Explain()**
- **Data Sharding**



Performance optimization

Continuous Performance Optimization

- **Performance Testing Tools:** Utilizing performance testing tools, such as Lighthouse or JMeter, enables us to simulate various user scenarios and identify areas for improvement.
- **Regular Code Reviews**
- **Automated Testing**



Performance optimization

Testing:

Test your deployed application to ensure everything is working as expected. Check that your Angular frontend is accessible and interacts correctly with your Express.js backend and MongoDB database.

Monitor and Maintain:

Monitor your deployed application for performance, errors, and security vulnerabilities. Set up logging and monitoring tools to track application metrics and user interactions. Regularly update dependencies and apply security patches to keep your application secure.



Performance optimization

Scale if Necessary:

As your application grows, you may need to scale your deployment to handle increased traffic and load. Cloud platforms offer auto-scaling and load balancing features to help manage.

Version control and continuous integration.

Version control and continuous integration (CI) are essential practices in modern software development that help teams collaborate effectively, maintain code quality, and streamline the development process.

Version Control:

Version control systems (VCS) track changes to files and directories over time. They allow multiple developers to collaborate on a project simultaneously without overwriting each other's work.



Version control and continuous integration.

Continuous Integration:

Continuous integration is a development practice where developers frequently integrate their code changes into a shared repository, often multiple times a day. Each integration triggers automated builds and tests to detect and fix integration errors quickly.

Version control and continuous integration.

CI workflows typically involve the following steps:

Developers commit their changes to the version control system.
A CI server detects the new commit and pulls the latest code from the repository.

The CI server runs automated build scripts, compiles the code, and executes automated tests.

If the build and tests pass, the changes are considered integrated successfully.

If the build or tests fail, the CI server notifies the team, and developers can investigate and fix the issues.



Version control and continuous integration.

Benefits of continuous integration include:

Early detection of integration errors: CI detects integration issues as soon as changes are committed, reducing the risk of conflicts and bugs.

Faster feedback: Developers receive immediate feedback on the quality of their code changes, allowing them to address issues promptly.

Confidence in code quality: With automated tests running on every integration, teams have greater confidence in the stability and reliability of their codebase.

Streamlined release process: CI enables teams to release software more frequently and with greater confidence, leading to faster delivery of new features and bug fixes.

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.in